

Supplementary material

fastPLS: a memory-aware accelerated SIMPLS implementation for high-dimensional biomedical data with optional CUDA and Metal routes

Dupe Ojo[†], Alessia Vignoli[†], Stefano Cacciatore^{*}, Leonardo Tenori^{*}

[†]These authors contributed equally and share first authorship.

^{*}Co-corresponding authors: Stefano Cacciatore, stefano.cacciatore@icgeb.org; Leonardo Tenori, tenori@cerm.unifi.it.

Central quantitative evidence used archived fastPLS 0.99.25, Git commit

7887401b09e25f54a546a253c255741cb1ab48e5, from source archive SHA-256

604f74d72f5e9540f7378efd37f2ca6ed87ccb9e73b912211331a8cd97233481. The archive, scripts, input checksums, session information, generated tables, and file ledger are indexed in Table S15. The deposited NMR workflow is retained only as explicitly labelled historical context.

Route-level evidence is consolidated rather than repeated. Table S1 defines residency; Tables S7-S7b cover deterministic validation, execution ablation, and PLS-SVD/SIMPLS shape comparison; Tables S7c-S8 and Figure S1 report controlled scaling and approximate-solver status; Table S9 defines float32 capability; Tables S10-S13 report software, backend, NMR, and ImageNet results; and Table S15 records provenance. Table S6 defines the evidence-status terminology used throughout.

S1. Current implementation scope

Table S1. Stage-level CPU, CUDA, and Metal residency. ‘Hybrid’ denotes an accelerated path with one or more host-side operations.

Stage	CPU	CUDA	Metal	Residency / precision note
Core PLS fit	Compiled C++	GPU products/range; host small solve	MPS products; host direction update	CUDA/Metal hybrid; float32 smoke-tested
OPLS filter	Compiled C++	Host filter + CUDA core	Host filter + MPS core	Hybrid; float32 smoke-tested
Kernel PLS	Host kernel + C++ core	Host nonlinear Gram + CUDA core	Host nonlinear Gram + MPS core	Nonlinear K is $O(n^2)$; hybrid
Prediction	Compiled projection	CUDA projection; host model/result I/O	MPS projection; host post-processing	Transfers remain part of execution
LDA	Compiled covariance/Cholesky	GPU moments/solve/score	MPS projection + host solve	float32 CPU/CUDA agreement verified
10-fold CV	Compiled folds/fits/scoring	Host scheduler + sequential GPU folds	Compiled/hybrid fold fits	No R-level refit loop; no concurrent folds
Solver dispatch	IRLBA deterministic; rSVD qualified approximate	rSVD qualified approximate; IRLBA unavailable	rSVD experimental/hybrid; IRLBA unavailable	Unavailable solver/backend combinations stop explicitly

Stage	CPU	CUDA	Metal	Residency / precision note
Classification heads	Argmax and compiled LDA	Argmax and CUDA LDA	Argmax; Metal projection + CPU LDA	Metal LDA is hybrid; lda_ridge is deprecated and ignored

The double-precision CPU backend is the broadest reference implementation. Native float32 is selected automatically when input matrices are float::float32 objects and is available for route-specific PLS-SVD, SIMPLS, OPLS, and kernel PLS combinations on supported CPU, CUDA, and Metal builds. Windows now has a portable float-package CPU fallback for rSVD PLS-SVD, SIMPLS, and linear-kernel SIMPLS with argmax; it is not the native compiled single-precision path. Windows float32 OPLS, nonlinear kernel PLS, LDA, CUDA, and Metal combinations stop before fitting instead of silently converting to double.

The 0.99.25 namespace explicitly exports pls(), pls.single.cv(), pls.double.cv(), evaluate(), plot.permutation(), ViP(), fastsvd(), fastcor(), fastPLS_backend(), has_cuda(), and has_metal(). Registered predict() and plot() methods use the standard R generics. No PCA function or PCA class method is exported. Public model families are PLS-SVD, SIMPLS, OPLS, and kernel PLS; public classification heads are argmax and LDA. rSVD is the default approximate solver with qualified CPU/CUDA controls, while IRLBA is available only on CPU. The legacy lda_ridge argument is deprecated and ignored: supplying it warns, and it is absent from cross-validation tuning records. LDA instead uses the fixed relative-ridge sequence 10^-8, 10^-6, 10^-5, 10^-4, 10^-3, and 10^-2, advancing only after Cholesky failure.

Estimator identity is a dispatch invariant. A requested SIMPLS model is fitted with the SIMPLS core on CPU, CUDA, and Metal; the CUDA fused SIMPLS-LDA route supplies the SIMPLS method identifier to the native core and trains LDA from those SIMPLS scores. Label-aware PLS-SVD remains available only through an explicit PLS-SVD request. When the guarded CUDA SIMPLS response representation is too large, execution stops with an informative error instead of replacing the estimator. Benchmark code checks requested against executed estimator identifiers and records an estimator-mismatch error if they differ.

Table S1a. Feature comparison between the primary deterministic SIMPLS reference and fastPLS. A check indicates that the public route provides the feature; qualifications are stated explicitly.

Feature	pls::simpls.fit	fastPLS 0.99.25
One fit returns component prefixes	Yes	Yes; not claimed as novel
Minimum-output fit	stripped mode retains coefficients	Optional suppression of scores, loadings, fitted values, and variance summaries

Feature	pls::simpls.fit	fastPLS 0.99.25
Compact latent prediction	No documented compact latent model	Yes; blocked prediction without a dense coefficient/prediction path
Implicit $X^T Y$ operator	No	Yes; optional and primarily memory-oriented
Truncated solvers	Reference SIMPLS direction calculation	Fixed-control CPU IRLBA or approximate rSVD
Accelerator routes	No native CUDA/Metal route	Route-specific CUDA and Metal; hybrid/experimental status remains explicit

S2. Numerical algorithms

Notation follows the main text: a indexes the current latent component, A is the retained component count, K is the number of cross-validation folds, and r is the target rank of an SVD calculation. Lowercase k is used only for retrieval or evaluation cutoffs, such as nearest-neighbour neighbourhood size and top- k accuracy, and never for PLS component indexing. The public R argument `ncomp` supplies A .

S2.1 PLS-SVD

Let centred predictor and response matrices be $X \in \mathbb{R}^{n \times p}$ and $Y \in \mathbb{R}^{n \times q}$. PLS-SVD computes dominant singular directions of

$$S = X^T Y.$$

The decomposition is requested once at the maximum valid component count. Prefixes are reused to produce all component numbers requested by the user. The maximum effective rank is bounded by $\min\{\text{rank}(X), \text{rank}(Y)\}$ and by the numerical rank of S . For a centred C -class indicator response, the columns sum to zero and $\text{rank}(Y) \leq C - 1$; the public factor-response path therefore caps PLS-SVD at $C - 1$ components. The model stores compact latent factors for prediction and omits a full coefficient cube when its estimated size exceeds the internal storage threshold and latent prediction is available.

Algorithm S1. PLS-SVD component-path construction.

1. Centre and, if requested, scale X with training statistics; centre numeric or indicator-coded Y .
2. Represent $S = X^T Y$ explicitly or as a product operator.
3. Compute the leading A singular triplets once, where A is the largest valid requested component count.
4. For each requested $a \in C$, form the compact latent prediction factors from prefixes $1, \dots, a$.

5. Store only the requested coefficient or prediction summaries; use blocked latent prediction when a dense coefficient path would exceed its memory threshold.

S2.2 SIMPLS

The SIMPLS estimator follows de Jong's sequential construction. For component a , the implementation obtains a dominant left direction r_a from the current cross-covariance state, forms and normalizes the score $t_a = X r_a$, computes predictor and response loadings, orthogonalizes the predictor loading against the preceding SIMPLS basis, and applies rank-one deflation. The default implementation accepts one refreshed direction after every deflation step.

The following execution optimizations are active in the current standard path:

1. Each randomized SIMPLS direction is generated from a fresh oversampled range sketch of the current deflated cross-covariance operator.
2. The row product $v_a^T S_a$ is evaluated once and reused in the rank-one deflation.
3. When its setup and storage can be amortized, the implementation caches the predictor cross-product and initial predictor-response cross-covariance so that score normalization and loadings avoid repeated observation-space products.
4. The coefficient matrix is updated as $B_a = B_{a-1} + r_a q_a^T$.
5. When fitted values are requested, they are updated as $\hat{Y}_a = \hat{Y}_{a-1} + t_a q_a^T$.
6. As in `pls::simpls.fit`, one fit supplies the standard sequential path through all components up to the requested maximum; this is not claimed as a fastPLS novelty.

A minimally optimized compiled baseline forms the cross-covariance explicitly, performs a fresh leading-direction solve and the standard de Jong score/loading/orthogonalization/deflation update for every component, and reconstructs requested dense coefficients and fitted values from retained latent factors. fastPLS changes this execution, not the estimator: it caches the deflation row product; updates coefficients and fitted values incrementally; conditionally caches X-transpose-X; retains compact latent factors when dense outputs are

unnecessary; and uses implicit cross-covariance products when predictor-response storage is limiting. The public CPU, CUDA, and Metal paths request one newly computed direction from the current deflated state for every component. Cross-component warm starts, multi-direction block refresh, and adaptive refresh policies were rejected and removed.

Algorithm S2. Accelerated sequential SIMPLS.

1. Centre and optionally scale X and centre Y ; initialize $S = X^T Y$, empty bases R , Q , and V , and the coefficient accumulator $B = 0$.
2. For component a , obtain one dominant left direction of the current deflated cross-covariance operator. IRLBA performs a fresh deterministic iterative solve. rSVD forms a fresh Gaussian sketch with target width one plus oversampling, applies the requested power iterations, performs QR orthonormalization, and extracts the leading vector from the reduced decomposition.
3. Form $t_a = X r_a$, normalize t_a and r_a , and calculate $p_a = X^T t_a$ and $q_a = Y^T t_a$.
4. Orthogonalize p_a against the existing V basis, reorthogonalize once, and normalize the resulting v_a .
5. Evaluate $h_a = v_a^T S_a$ once and update $S_{a+1} = S_a - v_a h_a$.
6. Append r_a , q_a , and v_a ; update $B_a = B_{a-1} + r_a q_a^T$ and, when requested, $\hat{Y}_a = \hat{Y}_{a-1} + t_a q_a^T$.
7. Snapshot outputs only at requested component counts. The optional $X^T X$ cache changes how score norms and loadings are evaluated, not the quantities defined above.

Table S2. Dominant computational and storage terms. Here, A is the retained number of PLS components and l is the rSVD sketch width (requested rank plus oversampling). Exact constants depend on matrix shape, solver convergence, and backend.

Path	Dominant work	Additional large storage	Memory-saving mechanism
Explicit PLS-SVD	Form $S = X^T Y$: $O(npq)$, then one truncated decomposition	S : $O(pq)$; latent factors $O((p+q)a)$	One decomposition supplies every requested component prefix
Implicit PLS-SVD	For sketch width l , $S\Omega = X^T(Y\Omega)$ and $S^T Q = Y^T(XQ)$	Sketches/factors $O((p+q)l)$; no S	Matrix-free products; label-aware class sums avoid dense indicator Y
Minimally optimized compiled SIMPLS	Form S : $O(npq)$; then A fresh direction solves and $O(A[np+nq+pq]+pA^2)$ sequential work, excluding solver cost	S and bases: $O(pq+(2p+q)A)$; dense requested prefixes: $O(C_l(pq+nq))$	Reference execution baseline; the standard path through components $1, \dots, A$ is not a fastPLS novelty
OPLS	Explicit $S = X^T Y$ and one-vector filter SVD, then SIMPLS predictive core	Filter S : $O(pq)$; bases $O(p \cdot \text{north})$ plus inner-model storage	No dense deflated X copy; the current orthogonal-filter stage remains explicit

Path	Dominant work	Additional large storage	Memory-saving mechanism
Nonlinear kernel PLS	Kernel construction plus SIMPLS in sample space	Centred Gram matrix $O(n^2)$	Blocked test prediction only; nonlinear kernel storage remains quadratic
Compact prediction	XtestR followed by latent response mapping	Test-score block and latent factors, not all $p \times q$ coefficient prefixes	Row-block streaming and low-rank latent mapping
fastPLS accelerated			Reuses sequential quantities and supports compact prediction;
SIMPLS	Incremental path: $O(Apq+Anq)$ output updates; compact prediction: $O(n_{\text{test}} A[p+q])$; implicit operator pair of width l : $O(n[p+q]l)$	Compact factors: $O((p+q)A)$; implicit work arrays: $O((n+p+q)l)$, without $O(pq)S$; optional $X^T X$ cache: $O(p^2)$	optional caching and implicit products trade setup or operator work for lower repeated work or storage

S2.3 OPLS

OPLS estimates orthogonal scores and loadings from X , removes variation that is orthogonal to the response-associated direction, and fits the predictive SIMPLS core to the filtered matrix. Prediction applies the stored training filter before the predictive model. The current CUDA path performs the OPLS filter on the CPU and the predictive SIMPLS stage on CUDA. Metal accelerates the larger filter products but retains selected host-side operations.

S2.4 Kernel PLS

The linear kernel dispatches to the ordinary linear SIMPLS route, avoiding an unnecessary n -by- n Gram matrix. RBF and polynomial kernels construct and centre an explicit training Gram matrix and apply stored training means to held-out kernels. One float64 Gram matrix requires $8n^2$ bytes before copies and workspaces: 0.75 GiB at $n=10,000$, 2.98 GiB at $n=20,000$, 6.71 GiB at $n=30,000$, and 18.63 GiB at $n=50,000$. Kernel centring, temporary products, and the R process require additional memory. The deterministic nonlinear-kernel validation included $n_{\text{train}}=42-180$; it establishes numerical agreement in that range, not large-sample feasibility. Conservative planning limits for the tested hardware are $n \leq 20,000$ on the 32-GB workstation and $n \leq 10,000$ on the 8-GB unified-memory Mac. These are operational ceilings based on reserving memory for multiple dense

buffers, not measured performance limits. Larger nonlinear-kernel jobs were not validated and require an explicit memory calculation. The current CUDA nonlinear-kernel route is host assisted, so the 16-GB device does not remove the host-memory constraint.

S2.5 Randomized SVD

For a linear operator M and target rank r , rSVD draws a Gaussian matrix, forms a range sketch, applies alternating products with M and its transpose, orthonormalizes the range, and decomposes the reduced matrix. Archived release 0.99.25 uses oversampling 20 and two power iterations as the effective CPU/CUDA default. This configuration met all predefined checks in the five-seed qualification panel; qualification applies to the controls and audited panel, not every future fit. Explicit unqualified controls warn and are stored in diagnostics. CUDA and Metal offload the large matrix products, but neither backend should be described as universally device-resident. In the current float32 CUDA implementation, QR and the reduced decomposition are finalized in host float32 after GPU range products. Metal likewise retains selected host-side reduced operations and SIMPLS direction updates. Supplementary Table S1 records residency by stage rather than assigning a single label to an entire model family.

S2.6 Bundled IRLBA

The float64 CPU route bundles augmented implicitly restarted Lanczos bidiagonalization. Under fixed inputs, starting state, and controls it is the deterministic numerical reference used here, but it remains an iterative truncated solver rather than an exact dense decomposition. It expands and restarts a bidiagonal Krylov subspace to approximate the requested dominant singular triplets. The default public controls are $\text{maxit}=1000$, $\text{tol}=1\text{e-}5$, $\text{eps}=1\text{e-}9$, and $\text{svtol}=1\text{e-}5$ unless overridden through Exact fallback is used only when either input dimension is below six. IRLBA is otherwise retained even when the requested rank approaches the smaller matrix dimension. The native float32 CPU and Metal routes use separate single-precision IRLBA-style implementations; CUDA float32 currently supports rSVD only.

S2.7 Implicit cross-covariance products

When $S = X^T Y$ is expensive to store, the SVD iteration can apply the same operator through

$$S V = X^T (Y V), S^T U = Y^T (X U).$$

For class labels $y_i \in \{1, \dots, C\}$, multiplication by one-hot Y is replaced by indexing, and multiplication by Y^T is replaced by class-wise sums. With the centred indicator response, the explicit class-sum identity is

$$S_{:,c} = \sum_{i: y_i=c} x_i - \frac{n_c}{n} \sum_{i=1}^n x_i.$$

The same centring and scaling values are applied in every row block. No independent SVD is fitted to separate blocks, so the mathematical operator remains the full cross-covariance.

The package uses distinct internal activation rules for rSVD and IRLBA because their iteration and memory profiles differ. These rules are implementation controls, not user-facing statistical parameters.

S2.8 Compact and streamed prediction

For SIMPLS, a prediction at retained component count A is evaluated through compact latent factors rather than a dense coefficient matrix. Large test matrices are processed in row blocks and intermediate score matrices are released after use. This limits retained prediction state to the current block and low-rank factors; it does not introduce a separate SVD solver.

S3. Classification heads

S3.1 Argmax

Argmax applies PLS regression to the multivariate class response and selects the class with the largest predicted response score. It is the standard PLS-DA decoding rule and requires no separate classifier fit.

S3.2 Latent-space LDA

LDA is fitted to the PLS training scores. For class counts n_c and means μ_c , the pooled covariance is computed from moments:

$$\Sigma = \frac{T^T T - \sum_c n_c \mu_c \mu_c^T}{\max(1, n - C)}.$$

For each class, the system $\Sigma w_c = \mu_c$ is solved by Cholesky factorization and triangular solves. If factorization fails, the implementation sets $s = \text{tr}(\Sigma)/d$ when finite and positive, or $s=1$ otherwise, and tries $\Sigma + \rho s I$ for $\rho \in \{10^{-8}, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}\}$ in this fixed order. The sequence is a numerical fallback, not a tuned hyperparameter. The discriminant is

Fallback-use audit. In the fixed-score numerical study, all 144 candidate float32 LDA fits (72 CPU and 72 CUDA) succeeded at the first nominal relative regularization value, $\rho = 1\text{e-}8$; no escalation occurred and CPU/CUDA selected values did not differ beyond float32 representation. The archived public end-to-end workflow rows did not retain the selected fallback value, so invocation frequencies are not claimed for those older benchmarks. Future publication reruns must record ρ for every fitted prefix and backend.

$$\delta_c(t) = t^\tau w_c - \frac{1}{2} \mu_c^\tau w_c + \log(n_c/n).$$

The float32 CPU, CUDA, and Metal routes keep their supported PLS arithmetic in the input precision. CUDA LDA uses single-precision score, covariance, factorization, and prediction buffers; Metal presently uses accelerated score projection with host-side LDA factorization. A controlled CPU/CUDA/Metal screen verified identical decoded labels for all four PLS families on a fixed classification task and relative prediction differences no larger than 2.4×10^{-6} on a fixed univariate-regression task.

S4. Cross-validation and metrics

`pls.single.cv()` evaluates eligible combinations of component count and prediction settings within one K-fold layer; `pls.double.cv()` adds an outer layer for performance estimation after tuning. For classification, auto selection uses accuracy; balanced accuracy, the unweighted mean of class-specific recalls, must be requested when class imbalance makes overall accuracy inappropriate. Dummy-response Q^2 is also available but is not decoded-label performance. For regression, auto selection uses RMSD. The 'r2' selection metric denotes an observed-mean held-out R^2 , whereas 'q2' uses fold-training response means. Full-data training R^2 is descriptive and is never optimized. Nested permutation inference uses the same selected endpoint and its appropriate tail.

Grouped permutation and fold rules preserve complete equal-size constraint blocks; failed null fits are recorded and excluded from the successful-null count.

Two uses of component grids are distinguished. Benchmark trajectory figures evaluate fixed component counts on a fixed test set and are descriptive. Training-only model selection reports the best value within the evaluated grid. Among 46 evaluated family-dataset choices, 20 reached the upper boundary, four reached the lower boundary, nine were response-rank limited, and 13 were interior. Thus 33/46 choices were constrained and are treated as fixed computational workloads rather than unconstrained predictive optima. The complete status is stored in `benchmark_results/manuscript_revision_cycle48_20260726/component_selection_by_family.csv`. NMR was the central sensitivity analysis: its grid was extended to 300 components and the one-standard-error rule selected interior settings. Table S14 reports supporting repeated-partition sensitivity, explicitly separated from archived-release central claims.

For observed labels y_i and decoded predictions $\hat{y}_i$, accuracy is $n^{-1} \sum_i I(y_i = \hat{y}_i)$. Top- k accuracy replaces this indicator with membership of y_i among the k highest class scores. For a numeric response, training fit is summarized by

$$R^2 = 1 - \frac{\sum_{i,j} (y_{ij} - \hat{y}_{ij})^2}{\sum_{i,j} (y_{ij} - \bar{y}_j)^2},$$

whereas cross-validated prediction is summarized by

$$Q^2 = 1 - \frac{\sum_{i,j} (y_{ij} - \hat{y}_{ij}^{(-f(i))})^2}{\sum_{i,j} (y_{ij} - \bar{y}_j^{(-f(i))})^2}.$$

In the displayed R^2 equation, full-data training R^2 uses fitted responses and the mean of each complete training-response column. The selectable observed-mean cross-validated R^2 uses the same numerator-denominator form, but substitutes out-of-fold predictions and the mean of each complete observed-response column. In the displayed Q^2 equation, every held-out fold contributes its prediction sum of squares, while its denominator is centred on the

mean estimated from that fold's training observations; fold numerators and denominators are summed before forming Q^2 . Independent-test Q^2 instead uses the complete training-response mean. RMSD is the square root of the out-of-fold sum of squared errors divided by the number of predicted response entries. For PLS-DA, R^2 and Q^2 use dummy-coded responses and are not decoded-label accuracy. The public metrics\$definitions element records the convention used.

S5. Real datasets and preprocessing

The benchmark writes a machine-generated dataset manifest containing source, licence, checksum, object names, preprocessing, split seed, n_{train} , n_{test} , p , q , and component grid. Dataset dimensions are read from the prepared task objects rather than transcribed manually.

Split-unit qualification. The archived cross-dataset benchmark does not contain an auditable subject or donor grouping variable for every prepared object. Its held-out accuracies and any row-level binomial intervals therefore quantify conditional computational performance only. Cell-level rows in Retina, Tabula Muris, and CBMC CITE-seq are not treated as independent biological replicates. Biomedical generalization claims require donor- or subject-grouped splits through constrain; where such identifiers are unavailable, the analysis is explicitly labelled a row-level computational benchmark.

Table S3. Dataset dimensions represented by the current prepared benchmark objects. The released manifest is the primary machine-readable record and reports counts before and after each filtering stage. CBMC CITE-seq $q=10$ denotes the ten raw ADT count responses; its RMSD is reported in those assay-count units.

Dataset	Task	Prepared n train	Prepared n test	Prepared p	Prepared q
TCGA-HNSC methylation	Classification	520	58	782	2
CCLE	Classification	547	71	1,000	18
TCGA-BRCA	Classification	756	88	1,000	5
MetRef	Classification	773	100	375	22
TCGA Pan-Cancer	Classification	3,000	982	850	32
GTEx v8	Classification	3,000	797	1,000	32
Retina	Classification	22,402	22,406	50	12
Tabula Muris	Classification	50,043	50,059	50	32
CIFAR-100	Classification	50,000	10,000	768	100
PRISM	Multivariate regression	479	54	1,000	4,686
NMR	Multivariate regression	1,200	321	13,000	28,355
CBMC CITE-seq	Multivariate regression	7,755	862	1,000	10
ImageNet/DINOv2	Exploratory classification/retrieval stress test	1,000,000	281,167	1,024	1,000

S5.1 MetRef

MetRef comprises longitudinal urine proton-NMR spectra from 22 donors. Zero-signal variables are removed, spectra are normalized and scaled with training-derived statistics, and donor is the class label. The manifest identifies the normalization function and retained bins.

S5.2 CIFAR-100 and ImageNet

CIFAR-100 and ImageNet were evaluated as precomputed image embeddings rather than by training image encoders. CIFAR-100 used its standard 50,000/10,000 partition [30]. The historical ImageNet archive contained 1,281,167 rows, 1,024 DINOv2 features, and 1,000 labels [28,29], but no primary canonical train/validation flag. Its exact DINOv2 checkpoint, image preprocessing, pooling rule, extraction script, and independently auditable image-to-row mapping were not retained. The stored feature matrix and labels can be checksummed and the downstream fastPLS analysis rerun, but the representation cannot be regenerated independently from the available metadata. Feature extraction was completed before the fastPLS benchmark and excluded from timing.

ImageNet is retained as a historical, partially reproducible foundation-model embedding stress test, not as a biomedical validation set. Its relevance is the post-extraction matrix regime: computational pathology models such as UNI and Prov-GigaPath produce large dense tile or slide representations for supervised downstream analysis [31,32]. The experiment tests whether fastPLS can process a matrix of that scale. It does not establish natural-to-medical-image transfer, diagnostic performance, or representation-level reproducibility.

S5.3 GTEx, TCGA, and CCLE

GTEx v8 tissue labels, TCGA Pan-Cancer disease labels, TCGA-BRCA PAM50 subtypes, TCGA-HNSC sample type, and CCLE lineage labels define classification outcomes. Outcome eligibility filtering and sample matching occur before the split; predictor imputation, zero-variance filtering, normalization, scaling, and any data-derived feature selection are estimated from training data only. Dataset-specific minimum-class rules and the number of removed observations are recorded in the manifest. Legacy descriptions such as “the first 1,000 features” denote a target maximum rather than the final matrix width: matching, finite-value checks, and zero-variance removal can reduce p , explaining prepared widths such as 850 or 782. The ordered retained identifiers, not the source-file column positions alone, define the reproducible feature set.

S5.4 CBMC CITE-seq, Retina, and Tabula Muris

CBMC CITE-seq [21] used the prepared SeuratData CBMC object (dataset version 3.1.4; 8,617 matched cells). X was the stored untransformed integer RNA assay-count matrix restricted before benchmarking to 1,000 preselected genes. Y was the stored untransformed integer ADT assay-count matrix for CD3, CD4, CD8, CD45RA, CD56, CD16, CD11c, CD14, CD19, and CD34. The stored split contained 7,755 training and 862 held-out cells and was loaded unchanged. The benchmark loader applied no library-size normalization, log transformation, centred-log-ratio transformation, or additional feature selection. The PLS call used mean centering without variance scaling: training means were subtracted from X and Y, held-out X was centered with the training X means, and the training Y means were restored to predictions. Consequently, the reported global RMSD is the square root of the mean squared error across all 8,620 held-out response entries (862 cells x 10 markers), in original ADT assay-count units per cell. The marker scales are heterogeneous, so this pooled RMSD is weighted toward high-count markers and is not numerically comparable with RMSD measured on normalized responses in another dataset. For the selected CUDA SIMPLS result, the global RMSD of 1,056.461 combined marker-wise RMSE values from 56.4 for CD34 to 3,191.3 for CD45RA. Retina and Tabula Muris are separate single-cell classification benchmarks and are named separately throughout the released manifest.

S5.5 PRISM and NMR

PRISM matches molecular profiles of cancer cell lines to multivariate drug-response measurements. NMR predicts a high-dimensional response spectrum from the input spectral representation. Both are multivariate regression tasks and are evaluated by RMSD rather than classification accuracy. The NMR source cohort and the prepared benchmark object are distinct: Table S3 reports the 1,521 matched, analysis-ready observations in the prepared object, not the complete source cohort. The released manifest must enumerate exclusions and train/test allocation so that these counts can be reconciled without inference. The NMR data are associated with the study of Vignoli, Cacciatore, and Tenori; source-study ethics and access conditions follow the cited source study.

S5.6 Redistribution and executable acquisition

Redistribution status was determined separately from source access status. A publicly downloadable source was not treated as permission to redistribute the processed benchmark matrix under the fastPLS licence. Only the

GPL-compatible breast and colon package examples, synthetic generators and generated synthetic results, and aggregate tables, figures, manifests, split indices, and checksums are redistributed. None of the prepared real-data matrices in Table S3 is bundled. The executable workflow benchmark/acquire_publication_datasets.R downloads primary public sources or validates user-supplied local files, writes paths, sizes, access classes, and checksums to acquisition_manifest.csv, and refuses silent release substitution. Exact commands and preprocessing contracts are documented in benchmark/DATA_ACQUISITION.md.

Table S3a. Redistribution and acquisition status. 'No' means that fastPLS does not redistribute the prepared real-data matrix, even when the upstream source is public. The acquisition command is run from the repository root; restricted validation requires the environment variable shown.

Dataset/object	Authoritative source	Access class	Prepared object redistributed?	Executable route
breast; colon	mixOmics; plsgenomics	GPL-compatible package data	Yes	data(breast); data(colon)
Synthetic and aggregate outputs	fastPLS generators/results	Project-generated	Yes	Run published benchmark scripts
MetRef	KODAMA::MetRef	Public R package	No	--dataset=metref
CIFAR-100	University of Toronto	Public download	No	--dataset=cifar100
CBMC CITE-seq	GEO GSE100866	Public repository	No	--dataset=cbmc_citeseq
Retina	GEO GSE63472; openTSNE pca_50	Public repositories	No	--dataset=retina
Tabula Muris	ExperimentHub EH1617	Public Bioconductor data	No	--dataset=tabula
GTEEx v8	GTEEx open access via UCSC Xena	Open expression/phenotype	No	--dataset=gtex_v8
TCGA tasks	TCGA open access via UCSC Xena	Open molecular/phenotype	No	--dataset=tcga_brca,tcga_hnsc_methylation,tcga_pan_cancer
CCLE	DepMap/CCLE 18Q2	Historical release	No	FASTPLS_CCLE_RDATA=... --dataset=ccele
PRISM	DepMap PRISM 19Q4	Release-controlled	No	FASTPLS_PRISM_RDATA=... --dataset=prism
NMR	Figshare share; MTBLS242/395/424	Source-study terms	No	FASTPLS_NMR_RDATA=... --dataset=nmr
ImageNet/DINOv2	ImageNet authorized copy; local extraction	Gated/non-commercial terms	No	FASTPLS_IMAGENET_RDATA=... --dataset=imagenet

The public acquisition command is Rscript benchmark/acquire_publication_datasets.R --dataset=<id> --out=<directory>. For ImageNet, NMR, CCLE, and PRISM, the listed environment variable points to a user-authorized local file. The workflow records a checksum but does not copy that file into the repository. GTEEx uses only open-access expression and phenotype data; protected sequence and full donor-level files are outside the benchmark. The ImageNet embeddings remain a derived object subject to the source-image terms and are not redistributed.

S6. Synthetic datasets and corresponding numerical results

Three predefined synthetic designs answered separate questions. The independent dense-reference panel covered ten conditions and 82 component prefixes: well-conditioned, nearly tied leading singular values, rank-deficient X

and Y , highly collinear predictors, $p < n$, $p > n$, high response dimension, a component requested at effective rank, multivariate regression, and dummy-response classification. A second formal comparison used five multivariate regression and three classification regimes with seeds 101, 202, and 303 for IRLBA and rSVD assessment. The separate controlled scaling design is reported in Section S12.2. Complete dimensions and generated matrices are reproducible from the repository scripts.

Classification regimes used balanced shuffled class labels and Gaussian class-specific latent prototypes. Each latent observation was its class prototype plus Gaussian noise with standard deviation 0.45; predictors were obtained through a Gaussian loading matrix with additional noise of standard deviation 0.08, and responses were one-hot class indicators. The three regimes covered $p < n$ with a low-rank response, $p > n$ with a higher-rank response, and near-collinear predictors. Every compared implementation received identical matrices, splits, component grids, folds, and seeds.

The dense-reference audit completed all 82 component prefixes without convergence failure. Across all cases, median and 95th-percentile coefficient errors were 8.50×10^{-16} and 3.39×10^{-15} ; corresponding held-out prediction errors were 7.62×10^{-16} and 2.72×10^{-15} . Maximum score/loading/projection subspace angles outside the intentionally near-tied case were 2.42×10^{-6} degrees, and classification label agreement was 1.000. For singular values separated by only 10^{-12} relatively, individual basis vectors were not identifiable; prediction error was 3.36×10^{-4} and the maximum subspace angle was 0.015 degrees. Orthogonality and deflation residuals remained below 6.36×10^{-15} and 8.67×10^{-16} . Separately, the IRLBA comparison comprised 117 component-level endpoints, all meeting predefined tolerances, with fixed-fold selection agreement in all tasks.

A historical single-seed development evaluation met 101/117 tolerance checks at oversampling 10 with one power iteration. Archived-release validation therefore used oversampling 20, two power iterations, and seeds 1, 7, 19, 43, and 123. All 585 CPU checks and all 40 CUDA checks met the predefined tolerances; rejected CUDA 10/1 and 10/2 controls met 0/40 and 27/40 checks. Component-level outputs are in `benchmark_results/frozen_release_0.99.25/simpls_estimator_preservation/` and `rsvd_cuda_reliability_final/`. These are solver-control reliability screens, not estimator-equivalence evidence.

S7. Benchmark execution and memory measurement

Each fit is run in an isolated R process. Total time is fitting plus prediction and excludes package installation, dataset acquisition and loading, package initialization, and metric calculation after prediction. The fit timer starts immediately before the public `pls()` call and stops after the fitted R model is returned. For CUDA routes, this interval includes host-side argument marshalling, host-to-device transfer of training matrices, device allocation, CUDA kernels and library calls, synchronization, and device-to-host transfer of model components exposed in the returned R object. The prediction timer starts immediately before `predict()` and stops after predictions or class scores are available in R. It includes host-to-device transfer of the test matrix and any model arrays not retained on the device, CUDA execution and synchronization, and device-to-host result transfer. Routes supporting resident workspaces reuse them; the benchmark does not add an artificial host round trip, but the public R model remains the interface between the separately timed fit and prediction calls. Therefore, reported CUDA time is end-to-end backend time rather than kernel-only time. After data and libraries are loaded, garbage collection is completed and host RSS and process-specific GPU memory are recorded immediately before fitting. The R process then signals a synchronized external sampler, which polls host RSS and PID-matched `nvidia-smi` device memory every 0.02 s throughout fitting and prediction. `/usr/bin/time` records the absolute maximum RSS over the complete isolated process. Incremental host memory is the maximum of fit-window samples and post-fit/post-prediction snapshots minus the pre-fit baseline; incremental process-level GPU memory is the PID-specific sampled peak minus its pre-fit baseline. Negative differences are truncated at zero. CUDA contexts were not preinitialized, so a zero pre-fit GPU baseline does not remove context creation, CUDA-library handles, allocator pools, persistent device objects, data/model storage, or workspaces created by the measured call. The GPU increment therefore measures the end-to-end device footprint triggered by the workflow and is not an isolated algorithmic-workspace measurement. Isolating workspace would require CUDA-API instrumentation after persistent runtime state had been initialized, which was not performed. For the repeated external SIMPLS comparison, `/usr/bin/time` supplied the complete-worker high-water RSS and the worker sampled its current RSS after loading and garbage collection, immediately before the fit. Their difference is reported as a baseline-corrected process peak increment. Because the absolute high-water mark spans the worker lifetime, the increment

can retain earlier loading high-water effects; it is therefore a conservative complete-process measure rather than fit-window workspace. Formula-based sizes of the cross-covariance, coefficient, fitted/residual, score, and prediction objects are reported alongside measured RSS.

Current-version central analyses used the frozen 0.99.25 execution archive and recorded its SHA-256, package version, input checksum, script identity, compiler, BLAS/LAPACK, thread settings, accelerator libraries, seed, rSVD controls, repetition, and status. NMR was rerun end to end from the prepared predictors and responses. For ImageNet, only the downstream PLS fitting and prediction stages were rerun; feature extraction remains historical and partially reproducible. Table S15 maps each claim to its result directory and provenance scope.

Table S4a. Exact reproducibility environment for the primary CUDA workstation and the Apple Metal validation system. Values were read from benchmark manifests, sessionInfo(), R configuration, package installation logs, linked runtime libraries, and operating-system queries. The CUDA and Metal columns are not direct hardware comparisons.

Item	CUDA workstation	Metal validation system
Platform	Intel Core i7-13700 (16 cores, 24 logical CPUs); 32,526,480 kB RAM; NVIDIA GeForce RTX 5060 Ti, 16,311 MiB VRAM	macOS 14.5 (23F79); Apple M3 (8 CPU cores, 10 GPU cores); 8,589,934,592 bytes unified memory; Metal 3
R and fastPLS	R 4.6.0; frozen central analyses used fastPLS 0.99.25 from commit 7887401b09e2. See Table S15 for the archive checksum and analysis mapping.	R 4.6.0; Metal diagnostics used development source states and are archival, not frozen central evidence. See Table S15.
Core R packages	Rcpp 1.1.1-1.1; RcppArmadillo 15.2.7-1; RcppEigen 0.3.4.0.2; Matrix 1.7-5; float 0.3-3; pls 2.9-0	Rcpp 1.1.2; RcppArmadillo 15.4.0-1; RcppEigen 0.3.4.0.2; Matrix 1.7-5; float 0.3-3; pls 2.9-0
External comparison packages	mdatools 0.15.0; chemometrics 1.4.4; pcv 1.1.0; plsdepot 0.3.1; plsgenomics 1.5-3; mixOmics 6.36.0; spls 2.3-2; ropls 1.44.0	Not used for the external-package comparison
C/C++ compiler	GCC/G++ 11.4.0; C++17	Homebrew Clang/Clang++ 22.1.1; C++17; macOS SDK 15.2
Effective C/C++ flags	-std=gnu++17 -fPIC -g -O2 -ffile-prefix-map=/build/r-base-gN72Ro/r-base-4.6.0=-fstack-protector-strong -Wformat -Werror=format-security -Wdate-time -D_FORTIFY_SOURCE=2; package definitions: -DRANN -DARMA_64BIT_WORD=1 -DFASTPLS_HAS_CUDA -DFASTPLS_HAS_CUDA_KERNELS	-std=gnu++17 -fPIC -falign-functions=64 -Wall -g -O2; Objective-C++: -x objective-c++ -fobjc-arc; package definitions: -DRANN -DARMA_64BIT_WORD=1 -DFASTPLS_HAS_Metal
Accelerator build/link	NVCC 13.0.88: -std=c++17 --extended-lambda --expt-relaxed-constexpr -DFASTPLS_HAS_CUDA -DFASTPLS_HAS_CUDA_KERNELS -Xcompiler -fPIC; linked with -lcudart -lcublas -lcusolver -lcublas	Linked frameworks: Foundation, Metal, and MetalPerformanceShaders
BLAS/LAPACK	Reference BLAS 3.10.0; LAPACK 3.10.0	R BLAS (/Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib); R LAPACK 3.12.1 (libRlapack.dylib)
CPU thread setting	1 effective BLAS thread; reference BLAS and no OpenMP; OMP_NUM_THREADS, OPENBLAS_NUM_THREADS, and MKL_NUM_THREADS unset	1 effective R-BLAS thread; no OpenMP; OMP_NUM_THREADS and VECLIB_MAXIMUM_THREADS unset
GPU runtime/libraries	NVIDIA driver 595.71.05; CUDA SDK 13.0.3; CUDA runtime 13.0.96; cuBLAS 13.1.1.3; cuSOLVER 12.0.4.66; cuRAND 10.4.0.35	Metal 3; Metal.framework and MetalPerformanceShaders.framework from macOS SDK 15.2

Table S4b. External implementation scope. Exact package versions and execution errors are captured by the run manifest.

Package	Functions evaluated where compatible	Comparison scope
pls	pls(method="simpls"), simpls.fit	SIMPLS reference for supported regression or dummy-response tasks
mdatools	pls, plsda	PLS regression or PLS-DA when the installed API supports the task

Package	Functions evaluated where compatible	Comparison scope
plsdepot	simpls	Independent SIMPLS fit
pcv	simpls	Independent SIMPLS fit where response shape is supported
plsgenomics	pls.regression	Genomics-oriented PLS regression/classification where installable
chemometrics	pls_eigen, pls2_nipals	Eigen- and NIPALS-based PLS references
mixOmics	plsda, splsda	PLS-DA and sparse PLS-DA; model differences are labelled
spls	spls, splsda	Sparse estimators; not treated as identical to dense SIMPLS
ropls	opls	PLS/OPLS where task type and class-count restrictions permit

Only estimator/task combinations with compatible outputs are used for speed-and-prediction comparisons. Estimator-matched tables require the same preprocessing, response representation, component count, prediction head, precision, thread setting, and timed work. A separate complete-workflow table may compare different estimators or heads, but is not interpreted as an implementation-only speed test. Unsupported multivariate responses, binary-only restrictions, timeouts, memory kills, and numerical failures remain in the final table rather than being dropped. Adapter errors are corrected before comparison or labelled as benchmark-code failures; they are not described as limitations of the external package.

S8. Backend reproducibility

Reproducibility experiments use identical data, preprocessing, folds, requested components, and seeds for the configured CPU, CUDA, and Metal routes. CPU builds delegate dense linear algebra to the BLAS/LAPACK implementation linked to R and can use OpenBLAS where it is installed. That build capability is recorded separately from measured performance. The primary software-comparison runs used one effective BLAS thread. No controlled one-thread versus multiple-thread scaling experiment was conducted, and no multicore speed-up is inferred. Because rSVD is stochastic and nearly tied singular values can rotate an equivalent basis, raw component columns are sign-aligned only for descriptive comparisons. Primary endpoints are prediction agreement, absolute predictive-metric difference, relative Frobenius prediction error, selected component count, numerical failures, and run-to-run variability. Principal angles compare latent subspaces.

Table S5. Backend reproducibility endpoints.

Endpoint	Definition	Interpretation
Prediction agreement	Fraction of identical decoded labels, or relative Frobenius difference for numeric predictions	Primary check of model-output agreement under backend changes
Predictive-metric difference	Absolute difference in accuracy, Q^2 , or RMSD	Statistical rather than bitwise equivalence

Endpoint	Definition	Interpretation
Principal angles	Angles between fitted latent subspaces	Robust to sign changes and rotations of nearly tied singular vectors
Selected-component agreement	Equality of the component count chosen on fixed folds	Checks downstream reproducibility of tuning
Numerical failures	Non-convergence, Cholesky fallback, non-finite output, timeout, or memory failure	Retained explicitly rather than excluded

The completed fixed-score LDA validation used MetRef, CIFAR-100, Retina, and Tabula Muris. CPU and CUDA float32 predictions were compared at fixed component counts; classifier-level agreement is reported separately from upstream rSVD variation.

S9. ImageNet computational role

The role of this experiment is computational rather than biomedical. It supplies a large multiclass embedding matrix resembling the post-extraction data form produced by medical-image foundation models [31,32]. Because representation provenance is incomplete, it is interpreted only as a historical feasibility and memory-throughput demonstration.

S10. Evidence-status terminology

Table S6 defines one hierarchy for execution, residency, and numerical evidence. A route can execute successfully yet remain experimental or numerically discordant. Only routes meeting the stated numerical tolerances contribute to validated speed claims; unavailable and failed routes remain visible.

S11. Evidence organization

Each central result maps to one table or figure in Sections S12-S20. Expanded component paths, raw replicates, and machine-level diagnostics are indexed in the repository rather than repeated here.

Table S6. Evidence-status glossary used in the manuscript, figures, and Supplementary tables.

Term	Meaning	Reporting consequence
Available / unavailable	The requested route can / cannot be built and executed on the platform.	Unavailable routes stop before model allocation.
Native / hybrid	All principal stages remain on the named backend / one or more stages execute on the host.	Residency is reported separately from numerical status.
Deterministic reference	Fixed-input, fixed-control CPU IRLBA or dense-LAPACK validation route.	Used for confirmatory numerical comparison; not called an exact dense solver when IRLBA is used.
Qualified approximate	The rSVD route met every stated tolerance in the specified validation panel.	Controls, seed, version, and status accompany each result.
Concordant / discordant	Paired predictions and endpoint metrics do / do not meet the stated agreement tolerances.	Discordant routes are excluded from validated speed claims.
Validated	The family/backend/endpoint combination has direct numerical evidence within the stated scope.	Claims are limited to that scope and precision.
Experimental / hybrid	The route executes but evidence is incomplete or includes host-assisted stages.	Warnings and diagnostics are retained; confirmatory claims are avoided.
Rejected / failed	Controls failed numerical criteria / execution did not complete.	The row remains visible and cannot support performance claims.

Table S6a. Practical interpretation of the computational evidence and recommended fastPLS route.

Question	Observed trigger or constraint	Recommended action
PLS family	CPU matched shapes favored one-shot PLS-SVD; qualified CUDA shapes placed sequential SIMPLS near parity. PLS-SVD is response-rank limited.	Use training-only prediction to choose the estimator. Prefer PLS-SVD for a low-rank, one-shot path; use SIMPLS when sequential components or components beyond response rank are required.
Compact prediction	Large test n, large q, or many requested prefixes; up to 77.7% lower incremental RSS in the ablation.	Retain latent factors and block predictions. Dense coefficient, score, loading, or fitted paths should be requested only when scientifically needed.
Implicit products	Memory pressure from X-transpose-Y; slower below 32 MiB and near time-neutral at 32-64 MiB in the controlled CPU sweep.	Use primarily to reduce memory for large p by q products, especially high q. Keep the explicit route for small cross-covariance matrices unless profiling supports otherwise.
CUDA	First qualified crossover at n = 5,000 or p = 2,000 in separate one-factor sweeps on the tested NVIDIA system.	Benchmark the intended shape including transfer and synchronization. Interpret speed only when paired predictions meet numerical tolerances.
Metal	Host-assisted stages and all 20 diagnostic rSVD routes were numerically discordant on the tested Apple system.	Treat Metal rSVD as experimental; use deterministic CPU execution for confirmatory work until the route is qualified.
IRLBA or rSVD	Need for deterministic reproducibility, diagnostic failure, material seed variation, ill-conditioning, or small problems where approximation saves little time.	Use CPU IRLBA for confirmatory analysis. Use rSVD for exploratory acceleration or routine analysis only when case diagnostics and multi-seed sensitivity are satisfactory; retain controls and numerical status with the result.
rSVD seeds	Scientifically consequential approximate fit or a new matrix regime.	Run several predefined seeds and compare predictions, endpoint metrics, and subspaces. Do not choose a seed using held-out test performance; switch to IRLBA if conclusions vary.
Boundary component	The selected component is the largest value evaluated or is constrained by rank.	Report 'best within the evaluated grid'; extend the training-only grid where feasible or use a predefined parsimonious rule such as one standard error.
Nonlinear kernel PLS	The float64 Gram matrix alone is 8n-squared bytes: about 0.75 GiB at 10,000, 4.66 GiB at 25,000, and 18.63 GiB at 50,000 samples.	Restrict to moderate n after accounting for copies and workspaces. For larger n, use linear PLS or an externally validated kernel approximation.
Unsupported route	Unavailable backend/solver/family combination or unqualified approximate controls.	Unavailable combinations produce an error. Experimental or unqualified routes warn and record diagnostics; no silent estimator substitution is permitted.

S12. Deterministic estimator validation

Deterministic estimator comparison and approximate-solver agreement are separate questions. Table S7 first reports the independent dense-LAPACK de Jong reference panel, then the iterative IRLBA comparison with `pls::simpls.fit` and the independent OPLS/kernel checks. rSVD is excluded from estimator-preservation evidence and evaluated in Section S13. The dense panel reports distributions as well as maxima because worst values are dominated by the deliberately near-tied singular-value case.

Table S7. Primary float64 CPU estimator-validation summary. Dense-reference results cover 82 component prefixes in ten conditions; the text reports median and 95th-percentile errors in addition to the maxima below. Angles are in degrees.

fastPLS scope	Reference	Endpoint tolerance status	Selection agreement	Max pred. rel. error	Max coef. rel. error	Max angle	Max metric diff.	Conclusion
SIMPLS / compiled dense LAPACK	Independent de Jong updates / base LAPACK SVD	82 / 82 completed	Not a selection study	3.36e-04 (4.53e-15 excluding tied case)	3.70e-04 (5.24e-15 excluding tied case)	0.0150 (2.42e-06 excluding tied case)	0 decoded-label difference	Machine precision except the non-identifiable near-tied basis
SIMPLS / deterministic IRLBA	<code>pls::simpls.fit</code> / de Jong	117 / 117	24 / 24	1.09e-05	1.13e-05	0.0015	1.92e-06	Met deterministic tolerances
OPLS (north 1-3) and kernel PLS (8 settings)	Independent filter/kernel + <code>pls::simpls.fit</code>	66 / 66	66 / 66	9.48e-12	3.07e-10	2.41e-06	3.19e-12	Reliable in deterministic float64 CPU scope

S12.1 Archival execution ablation and direct PLS-family timing

These development-stage ablations predate the frozen central rerun and are retained only to document how the execution design was developed; they do not support release-level speed claims. Each ablation changed one internal feature relative to a minimally optimized compiled SIMPLS baseline while holding data, split, estimator, deterministic IRLBA solver, component count, seed, and prediction head fixed. Three isolated runs were used per configuration. The measured ranges quantify workload-dependent trade-offs rather than universal speed-ups.

Table S7a. Same-code SIMPLS execution ablation. All endpoint metric differences were zero and minimum classification agreement was 1.000.

Optimization	Datasets	Time speed-up range	Incremental RSS reduction	Interpretation
Cached deflation products	4	0.94-1.02x	0.16-70.60%	Memory benefit; no uniform time gain
Cached X-transpose-X	4 (active in 1)	0.99-1.00x	-0.06-0.57%	Shape-gated; negligible in this panel
Compact prediction	4	0.97-1.24x	-0.02-77.71%	Largest benefit for high-response PRISM
Incremental coefficients	4	0.99-1.07x	-3.66-2.68%	Small isolated effect
Implicit cross-covariance	4	0.065-6.24x	0.12-70.64%	Faster only in the high-response regime

The matched PLS-family timing study used the same synthetic matrices, requested components, rSVD controls (oversampling 10, one power iteration, seeds 101-103), split, and backend for PLS-SVD and SIMPLS. The two estimators can differ predictively; this experiment compares execution time only.

Table S7b. Direct matched-shape runtime comparison. Ratios are SIMPLS time divided by PLS-SVD time; values below one favor SIMPLS.

S12.2 Controlled scaling and route crossovers

The baseline regression scenario used 2,000 training and 400 test observations, $p = 400$, $q = 100$, rank 30, and 20 retained components. One factor varied at a time: $n = 500, 1,000, 2,000, 5,000, 10,000$; $p = 50, 100, 250, 500, 1,000, 2,000$; $q = 10, 25, 50, 100, 250, 500, 1,000$; retained components = 2, 5, 10, 20, 40, 80; requested prefixes = 1, 2, 5, 10, 20; exact cross-covariance rank = 5, 10, 20, 40, 80; classes = 5, 10, 25, 50, 100, 200; and explicit cross-covariance storage = 2, 4, 8, 16, 32, 64 MiB. The rank sweep used noise-free responses to preserve exact rank; other regression sweeps used fixed Gaussian response noise. Classification labels arose from class scores in a fixed latent space. Three matched data/solver seeds were used per point. Every scenario included deterministic float64 CPU IRLBA and automatic CPU/CUDA rSVD with oversampling 20 and two power iterations; explicit and implicit rSVD were both forced in the storage sweep.

All 486 CPU/CUDA runs completed. Automatic rSVD produced 73 tolerance failures among 276 runs; failures were retained and excluded from crossover inference. All 72 forced explicit/implicit runs met tolerance. Qualified CUDA speed crossovers occurred at $n = 5,000$, $p = 2,000$, and throughout the 2-64 MiB storage sweep. The CPU implicit route reduced incremental RSS by 29.7-47.5% but was slower below 32 MiB and near parity thereafter. CUDA explicit/implicit times differed by less than 1% through 32 MiB and by 0.1% at 64 MiB. The separate Metal smoke study completed 56 runs; all 20 Metal rSVD routes were outside tolerance, so no Metal performance crossover is claimed.

Table S7c. Controlled one-factor-at-a-time scaling. A point is qualified only when every replicate met the numerical tolerances; crossover values are the first tested values, not extrapolated thresholds.

Factor	Tested values	CPU qualified	CUDA qualified	First qualified CUDA crossover	Median total-time range, CPU / CUDA (s)
Classes	5, 10, 25, 50, 100, 200	3/6	3/6	-	0.02-0.528 / 0.238-0.295
Cross-covariance MiB	1.9989, 3.9978, 8.00323, 15.9988, 31.9977, 64.003	6/6	6/6	1.9989	0.933-9.314 / 0.307-1.561
Training observations	500, 1000, 2000, 5000, 10000	5/5	5/5	5000	0.071-1.175 / 0.26-0.325
Retained components	2, 5, 10, 20, 40, 80	0/6	0/6	-	0.132-0.58 / 0.251-0.349
Predictors	50, 100, 250, 500, 1000, 2000	6/6	6/6	2000	0.025-0.634 / 0.258-0.339
Requested prefixes	1, 2, 5, 10, 20	5/5	5/5	-	0.064-0.247 / 0.241-0.397
Responses	10, 25, 50, 100, 250, 500, 1000	7/7	4/7	-	0.031-0.915 / 0.25-0.366
Exact rank	5, 10, 20, 40, 80	3/5	3/5	-	0.25-0.252 / 0.254-0.257

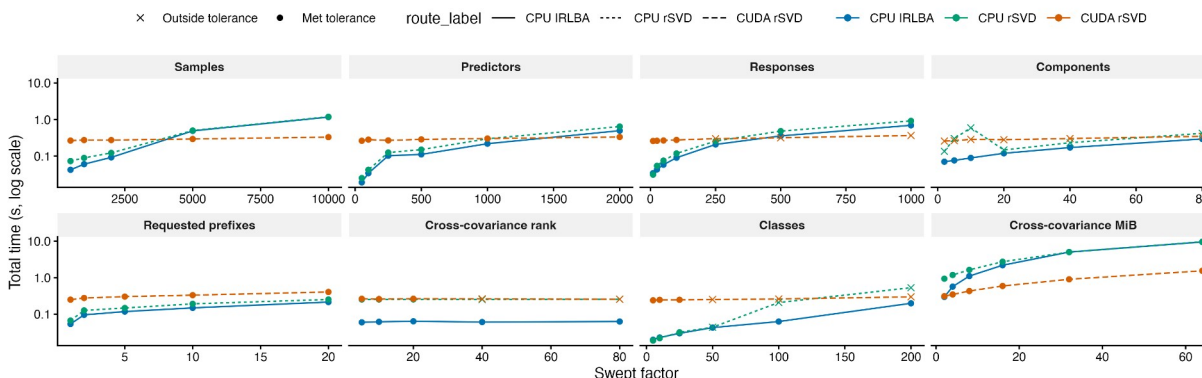
Table S7d. Forced explicit versus implicit cross-covariance routes. Incremental RSS excludes the loaded synthetic input but includes fit and prediction workspaces.

Backend	S size (MiB)	Explicit time (s)	Implicit time (s)	Implicit / explicit	Incremental RSS explicit / implicit (MiB)	Qualified time / memory preference
CPU	1.999	0.292	0.938	3.21	32.79 / 17.23	explicit / implicit
CUDA	1.999	0.304	0.302	0.993	376.5 / 376.5	implicit / explicit
CPU	3.998	0.565	1.16	2.06	46.84 / 25.23	explicit / implicit
CUDA	3.998	0.344	0.339	0.985	384.2 / 384.5	implicit / explicit
CPU	8.003	1.1	1.63	1.48	80.83 / 56.84	explicit / implicit
CUDA	8.003	0.419	0.416	0.993	410.2 / 410.3	implicit / explicit
CPU	16	2.16	2.69	1.25	136.8 / 85.24	explicit / implicit
CUDA	16	0.578	0.573	0.991	446.6 / 446.6	implicit / implicit
CPU	32	4.98	4.97	0.999	225.3 / 128.8	implicit / implicit
CUDA	32	0.897	0.89	0.992	485.5 / 497.7	implicit / explicit
CPU	64	9.3	9.36	1.01	427 / 261	explicit / implicit
CUDA	64	1.55	1.55	1	604.4 / 596.7	explicit / implicit

Archived-release controlled one-factor scaling
fastPLS 0.99.25; rSVD oversampling 20, two power iterations; 486/486 runs completed

A Controlled SIMPLS runtime scaling

Median of three isolated runs; open points are excluded from speed claims



B Baseline-corrected host-memory scaling

Complete-process increment, not isolated algorithm workspace

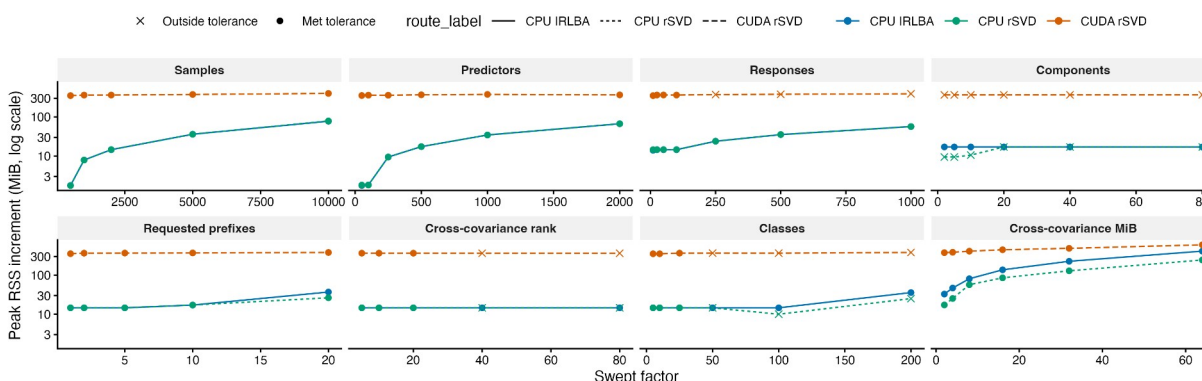


Figure S1. Archived-release controlled one-factor SIMPLS scaling. Median total time and baseline-corrected complete-process peak RSS are shown across sample, predictor, response, component, requested-prefix, rank, class-count, and cross-covariance-size sweeps. Open points were outside predefined numerical tolerance and are excluded from speed claims. Each point summarizes three isolated runs; all 486 requested runs completed.

Shape	n / p / q / A	CPU ratio	CUDA ratio	Interpretation
Wide	400 / 2000 / 20 / 10	1.18	0.93	Near parity
Tall-thin	5000 / 50 / 20 / 10	1.00	0.94	Near parity
High response	1000 / 300 / 500 / 50	1.11	0.94	Near parity
Balanced	5000 / 500 / 50 / 50	3.84	0.98	CPU favors PLS-SVD
High components	3000 / 768 / 200 / 100	1.34	0.92	CUDA near parity

S13. Approximate rSVD reliability

rSVD uses a Gaussian range sketch, oversampling, power iterations, and a recorded seed. Qualification required relative Frobenius prediction error ≤ 0.05 , prediction correlation ≥ 0.99 , each score/projection/loading subspace angle ≤ 10 degrees, decoded-label agreement ≥ 0.99 , and absolute predictive-metric difference ≤ 0.01 . These predefined values are computational non-inferiority screens, not clinical or exact-equivalence margins. Release validation used seeds 1, 7, 19, 43, and 123 and required every check to meet tolerance. The

oversampling-20, two-power configuration met 585/585 CPU and 40/40 CUDA checks in that panel; the broader controlled shape study identified out-of-tolerance regimes, which are reported separately and excluded from validated speed claims. Metal remains unqualified.

Table S8. rSVD numerical-validation summary for fastPLS 0.99.25. Release rows use seeds 1, 7, 19, 43, and 123; rejected alternatives were evaluated over the same seeds.

Backend	Oversample	Power	Tolerance checks met	Max pred. rel. error	Min pred. corr.	Max score angle	Min label agree.	Max metric diff.	Status
CPU	20	2	585/585	1.24e-09	1.00000	<0.001	1.000	5.32e-10	v0.99.25; Qualified release default
CUDA	10	1	0/40	0.2486	0.96884	NR	0.830	0.0300	v0.99.25; Rejected
CUDA	10	2	27/40	0.0822	0.99602	NR	0.940	0.0200	v0.99.25; Rejected
CUDA	10	4	40/40	0.0078	0.99996	NR	0.990	6.75e-05	v0.99.25; Qualified alternative
CUDA	20	1	40/40	0.0027	1.00000	NR	1.000	0.0002	v0.99.25; Qualified alternative
CUDA	20	2	40/40	6.33e-06	1.00000	NR	1.000	3.78e-07	v0.99.25; Qualified release default
CUDA	20	4	40/40	6.20e-06	1.00000	NR	1.000	1.05e-07	v0.99.25; Qualified alternative
Metal	20	2	NR	NR	NR	NR	NR	NR	v0.99.25; Unqualified; warning

Exploratory approximate-solver workflow speed

This one-power rSVD setting did not meet the complete numerical audit and is not used as estimator-preservation evidence.

C Workflow speed of the exploratory rSVD setting

Matched float64 CPU SIMPLS; oversample=10, power=1, seed=123; 101/117 audit checks met (not qualified)

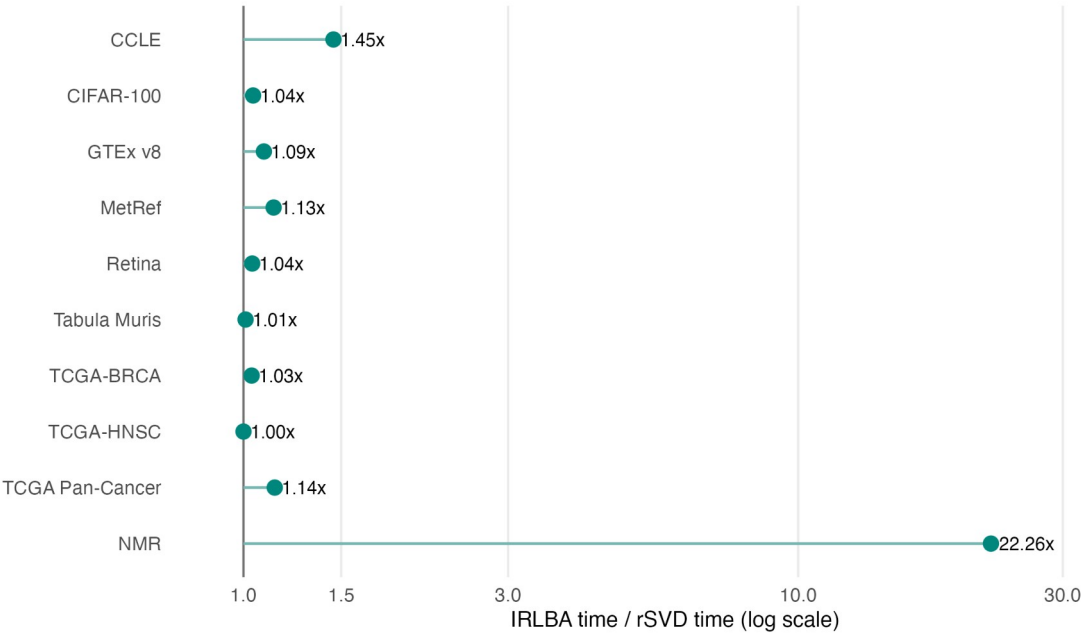


Figure S2. Historical exploratory one-power rSVD workflow speed relative to deterministic IRLBA. The setting used oversampling 10, one power iteration, and seed 123; it met only 101/117 checks and is excluded from estimator-preservation and release-default claims.

S14. float32 capability

float64 is the confirmatory reference. float32 input halves raw matrix storage but does not guarantee lower peak process memory, faster execution, or numerical agreement. Public calls allow validated routes, warn for experimental, hybrid, or measured-risk routes, and stop before allocation for unavailable routes.

Table S9. Primary float32 capability matrix. Endpoint statuses are validated, experimental, hybrid, unavailable, or failed; extreme-response status refers to $q \geq 10,000$ with at least 50 components.

Family	Kernel	Backend	Endpoint status	Solver	Residency	Windows	Extreme q
PLS-SVD	n/a	CPU	regression=validated; argmax=validated; LDA=validated	rSVD validated; IRLBA experimental	CPU	portable CPU rSVD regression/argmax; native LDA unavailable	experimental/not applicable
PLS-SVD	n/a	CUDA	regression=validated; argmax=validated; LDA=validated	rSVD only	device	unavailable	experimental/not applicable
PLS-SVD	n/a	Metal	regression=experimental; argmax=experimental; LDA=hybrid	rSVD and IRLBA experimental	Metal PLS scores plus CPU float32 LDA/device with hybrid reduced decomposition	unavailable	experimental/not applicable
SIMPLS	n/a	CPU	regression=validated; argmax=experimental; LDA=experimental	rSVD validated; IRLBA experimental	CPU	portable CPU rSVD regression/argmax; native LDA unavailable	failed/not applicable
SIMPLS	n/a	CUDA	regression=validated; argmax=experimental; LDA=experimental	rSVD only	Hybrid in float32 label- aware classification: host sequential SIMPLS and score projection; CUDA rSVD range products and LDA	unavailable	failed/not applicable
SIMPLS	n/a	Metal	regression=experimental; argmax=experimental; LDA=hybrid	rSVD and IRLBA experimental	Metal PLS scores plus CPU float32 LDA/device with hybrid reduced decomposition	unavailable	failed/not applicable
OPLS	n/a	CPU	regression=validated; argmax=validated; LDA=validated	rSVD validated; IRLBA experimental	CPU	unavailable	failed/not applicable
OPLS	n/a	CUDA	regression=hybrid; argmax=hybrid; LDA=hybrid	rSVD only	CUDA inner PLS with host- resident OPLS orchestration	unavailable	failed/not applicable
OPLS	n/a	Metal	regression=hybrid; argmax=hybrid; LDA=hybrid	rSVD and IRLBA experimental	Metal inner PLS with host- resident OPLS orchestration	unavailable	failed/not applicable
kernel PLS	linear	CPU	regression=validated; argmax=experimental; LDA=experimental	rSVD validated; IRLBA experimental	CPU	portable CPU rSVD regression/argmax; native LDA unavailable	failed/not applicable
kernel PLS	linear	CUDA	regression=validated; argmax=experimental; LDA=experimental	rSVD only	device	unavailable	failed/not applicable
kernel PLS	linear	Metal	regression=experimental; argmax=experimental; LDA=hybrid	rSVD and IRLBA experimental	Metal PLS scores plus CPU float32 LDA/device with hybrid reduced decomposition	unavailable	failed/not applicable
kernel PLS	nonlinear	CPU	regression=experimental; argmax=experimental; LDA=experimental	rSVD validated; IRLBA experimental	CPU with explicit Gram matrix	unavailable	failed/not applicable
kernel PLS	nonlinear	CUDA	regression=hybrid; argmax=hybrid; LDA=hybrid	rSVD only	CUDA kernel products with host-resident Gram centering/orchestration	unavailable	failed/not applicable
kernel PLS	nonlinear	Metal	regression=hybrid; argmax=hybrid; LDA=hybrid	rSVD and IRLBA experimental	Metal kernel products with host-resident Gram centering/orchestration	unavailable	failed/not applicable

S15. External software comparison

The repeated comparison used float64, deterministic CPU SIMPLS, identical splits and component counts, one effective BLAS thread, and a 10,000-s timeout. Package and data loading occurred before timing; no numerical warm-up was performed. Every method-dataset pair ran in three fresh R processes. Two profiles were kept separate. In the minimum-output profile, fastPLS retained the full coefficient path, means, and compact internal

prediction/audit factors, while `pls::simpls.fit(stripped = TRUE)` retained the full coefficient path and means; neither returned score, loading, fitted-value, residual, or variance arrays. In the public-workflow profile, `fastPLS` retained its ordinary compact object and variance summary, whereas `pls::simpls.fit` retained coefficients, X/Y scores and loadings, projection, fitted values, residuals, and X-variance quantities. Absolute lifetime peak RSS, the immediately pre-fit process baseline, and their difference were retained separately. The difference is a baseline-corrected complete-process increment, not isolated algorithmic allocation. The theoretical sizes of the largest retained dense objects were calculated from matrix dimensions and float64 element width. All 108 runs completed and all paired accuracies were identical. The older 126-run multi-package panel is retained as a workflow comparison only: 110 completed, 12 were package limitations, two timed out, and two errored. It is not labelled an estimator-kernel comparison because outputs and model families differ.

Table S10a. Repeated deterministic SIMPLS comparison. Times are median (IQR) seconds from three fresh processes. Speed-up is `pls::simpls.fit` divided by `fastPLS`; values above one favor `fastPLS`.

Dataset	Profile	fastPLS s	pls s	Speed-up	Accuracy fastPLS / pls	Completed reps
ccle	estimator kernel	0.079 (0.000)	0.082 (0.001)	1.04	0.7324 / 0.7324	3 / 3
cifar100	estimator kernel	10.632 (0.122)	12.902 (0.278)	1.21	0.8739 / 0.8739	3 / 3
gltx_v8	estimator kernel	0.242 (0.005)	0.359 (0.002)	1.48	0.9310 / 0.9310	3 / 3
metref	estimator kernel	0.027 (0.001)	0.024 (0.001)	0.89	0.7700 / 0.7700	3 / 3
retina	estimator kernel	0.411 (0.004)	0.141 (0.001)	0.34	0.9678 / 0.9678	3 / 3
tabula	estimator kernel	1.620 (0.017)	0.575 (0.006)	0.35	0.8006 / 0.8006	3 / 3
tcga_brca	estimator kernel	0.021 (0.001)	0.019 (0.001)	0.90	0.8523 / 0.8523	3 / 3
tcga_hnsc_methylation	estimator kernel	0.015 (0.001)	0.009 (0.000)	0.60	1.0000 / 1.0000	3 / 3
tcga_pan_cancer	estimator kernel	0.295 (0.008)	0.427 (0.001)	1.45	0.9114 / 0.9114	3 / 3

Table S10b. Output-materialization contract used for the repeated comparison.

Profile	fastPLS retained outputs	pls::simpls.fit retained outputs	Interpretation
Minimum outputs	Full coefficient path, means, compact prediction/audit factors; no scores, loadings, fitted arrays, or variance summary	stripped=TRUE: full coefficient path and means only	Closest available common prediction endpoint; reports numerical-kernel plus unavoidable implementation state
Public workflow	Ordinary compact fit object, means, latent prediction factors, variance summary; final predictions	Ordinary object with coefficient, score, loading, projection, fitted, residual, and variance paths; final predictions	Complete user workflow; time and memory include each package's normal output policy

Table S10c. Host-memory accounting for the minimum-output profile. Values are MiB; F/P denotes `fastPLS` / `pls::simpls.fit`. Absolute peak and corrected increment are median (IQR) across three fresh processes; pre-fit baseline and theoretical storage are medians or dimension-derived values. Both methods retain a $p \times q \times A$ coefficient path in this profile.

Dataset	Absolute peak F / P	Pre-fit baseline F / P	Peak increment F / P	Largest retained dense object F / P
ccle	474.3 / 491.8	397.9 / 397.8	76.5 / 94.0	coefficient path ($p \times q \times A$): 6.87 / coefficient path ($p \times q \times A$): 6.87
cifar100	1975.9 / 2033.9	1906.7 / 1906.8	69.2 / 127.1	coefficient path ($p \times q \times A$): 58.59 / coefficient path ($p \times q \times A$): 58.59

Dataset	Absolute peak F / P	Pre-fit baseline F / P	Peak increment F / P	Largest retained dense object F / P
gliex_v8	513.1 / 544.8	428.2 / 428.2	84.9 / 116.6	coefficient path (p x q x A): 7.81 / coefficient path (p x q x A): 7.81
metref	472.1 / 470.0	395.3 / 395.4	76.8 / 74.5	coefficient path (p x q x A): 1.38 / coefficient path (p x q x A): 1.38
retina	551.3 / 521.8	422.5 / 422.4	128.9 / 99.4	coefficient path (p x q x A): 0.23 / coefficient path (p x q x A): 0.23
tabula	667.8 / 645.9	478.4 / 478.4	189.4 / 167.5	coefficient path (p x q x A): 0.61 / coefficient path (p x q x A): 0.61
tcga_brca	469.7 / 481.0	399.5 / 399.6	70.1 / 81.4	coefficient path (p x q x A): 0.19 / coefficient path (p x q x A): 0.19
tcga_hnsc_methylation	465.6 / 470.6	396.2 / 396.2	69.4 / 74.3	coefficient path (p x q x A): 0.02 / coefficient path (p x q x A): 0.02
tcga_pan_cancer	513.2 / 547.3	425.3 / 425.4	87.8 / 121.9	coefficient path (p x q x A): 10.38 / coefficient path (p x q x A): 10.38

Table S10d. Host-memory accounting for ordinary public workflows. Values are MiB; F/P denotes fastPLS / pls::simpls.fit. Absolute peak and corrected increment are median (IQR) across three fresh processes. The largest retained dense object is the $p \times q$ final coefficient matrix for fastPLS and one $n_{\text{train}} \times q \times A$ fitted or residual path for pls::simpls.fit; the reference object ordinarily retains both fitted and residual paths. Measured RSS includes runtime, allocator, temporary, and output storage.

Dataset	Absolute peak F / P	Pre-fit baseline F / P	Peak increment F / P	Largest retained dense object F / P
ccele	490.0 / 496.7	397.9 / 397.8	92.1 / 98.9	final coefficient matrix ($p \times q$): 0.14 / fitted/residual response path ($n_{\text{train}} \times q \times A$ each): 3.76
cifar100	2493.6 / 13415.2	1906.8 / 1906.6	586.8 / 11508.8	final coefficient matrix ($p \times q$): 0.59 / fitted/residual response path ($n_{\text{train}} \times q \times A$ each): 3814.70
gtex_v8	599.7 / 641.0	428.2 / 428.2	171.5 / 212.9	final coefficient matrix ($p \times q$): 0.24 / fitted/residual response path ($n_{\text{train}} \times q \times A$ each): 23.44
metref	478.0 / 484.4	395.3 / 395.4	82.7 / 89.0	final coefficient matrix ($p \times q$): 0.06 / fitted/residual response path ($n_{\text{train}} \times q \times A$ each): 2.85
retina	564.2 / 896.3	422.5 / 422.5	141.7 / 473.8	final coefficient matrix ($p \times q$): 0.00 / fitted/residual response path ($n_{\text{train}} \times q \times A$ each): 102.55
tabula	705.7 / 2697.2	478.4 / 478.4	227.3 / 2218.8	final coefficient matrix ($p \times q$): 0.01 / fitted/residual response path ($n_{\text{train}} \times q \times A$ each): 610.88
tcga_brca	498.6 / 486.9	399.6 / 399.6	99.0 / 87.3	final coefficient matrix ($p \times q$): 0.04 / fitted/residual response path ($n_{\text{train}} \times q \times A$ each): 0.14
tcga_hnsc_methylation	480.9 / 473.7	396.2 / 396.2	84.7 / 77.4	final coefficient matrix ($p \times q$): 0.01 / fitted/residual response path ($n_{\text{train}} \times q \times A$ each): 0.02
tcga_pan_cancer	586.1 / 687.1	425.3 / 425.3	160.7 / 261.9	final coefficient matrix ($p \times q$): 0.21 / fitted/residual response path ($n_{\text{train}} \times q \times A$ each): 36.62

Repeated single-CPU SIMPLS workflows

fastPLS 0.99.25 versus pls 2.9.0; deterministic float64 routes

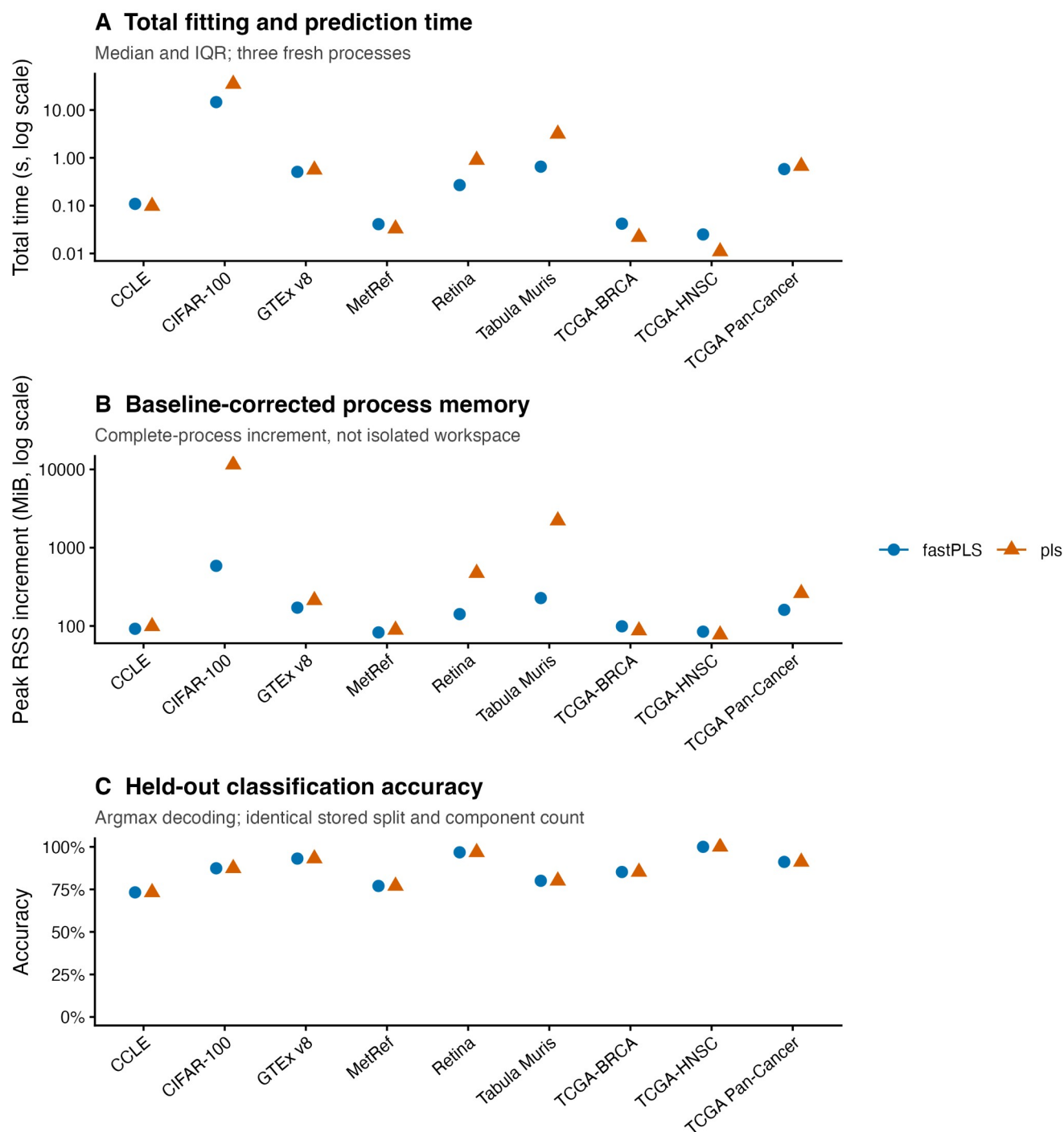


Figure S3. Repeated deterministic float64 single-CPU SIMPLS public workflows with fastPLS 0.99.25 and pls 2.9.0. Panels show median total fitting-plus-prediction time, baseline-corrected complete-process peak RSS, and held-out argmax accuracy. Error bars are IQRs from three fresh processes per method-dataset pair. Splits and component counts were identical, and accuracy was identical for every pair. Full values and the separate minimum-output comparison are reported in Tables S10a-S10d.

Table S10e. Broad external-package workflow comparison. Cells report accuracy; total fitting-plus-prediction time; and absolute peak process RSS. Implementations retain package-specific outputs, and unsupported runs, timeouts, and errors remain explicit.

Dataset	A	fastPLS argmax	fastPLS LDA	Fastest external	Best-accuracy external
CCELE	50	0.7324; 0.077s; 469MB	0.7746; 0.091s; 470MB	pls/SIMPLS: 0.7324; 0.109s; 135MB	pls/genomics/PLS-LDA: 0.7746; 0.114s; 172MB
GTEX v8	32	0.9310; 0.234s; 509MB	0.9611; 0.274s; 498MB	pls/genomics/PLS-LDA: 0.9611; 0.360s; 203MB	pls/genomics/PLS-LDA: 0.9611; 0.360s; 203MB
MetRef	22	0.7700; 0.034s; 470MB	0.8800; 0.037s; 470MB	pls/SIMPLS: 0.7700; 0.037s; 125MB	pls/genomics/PLS-LDA: 0.8800; 0.038s; 146MB
Retina	50	0.9678; 0.084s; 530MB	0.9691; 0.118s; 517MB	chemometrics/PLS eigen: 0.9344; 0.136s; 163MB	pls/genomics/PLS-LDA: 0.9691; 0.760s; 289MB
Tabula Muris	50	0.8006; 0.346s; 682MB	0.8752; 0.309s; 596MB	chemometrics/PLS eigen: 0.7833; 0.605s; 314MB	pls/genomics/PLS-LDA: 0.8752; 2.092s; 492MB
TCGA- BRCA	5	0.8523; 0.030s; 471MB	0.8750; 0.032s; 470MB	pls/genomics/PLS-LDA: 0.8750; 0.021s; 145MB	chemometrics/PLS eigen: 0.8864; 1.036s; 180MB
TCGA-HNSC methyl.	2	1.0000; 0.023s; 467MB	0.9828; 0.023s; 467MB	pls/genomics/PLS-LDA: 0.9828; 0.010s; 133MB	pls/SIMPLS: 1.0000; 0.013s; 107MB
TCGA Pan- Cancer	50	0.9114; 0.263s; 500MB	0.9155; 0.323s; 495MB	pls/genomics/PLS-LDA: 0.9155; 0.533s; 221MB	pls/genomics/PLS-LDA: 0.9155; 0.533s; 221MB
CIFAR-100	100	0.8739; 8.189s; 1688MB	0.8710; 10.118s; 1687MB	pls/SIMPLS: 0.8739; 34.647s; 13090MB	pls/SIMPLS: 0.8739; 34.647s; 13090MB

S15.1 Cross-language IKPLS software comparison

This archived-release comparison is not estimator matched. IKPLS 6.1.2 implements Dayal-MacGregor Improved Kernel PLS in two formulations: the score-explicit variant computes and retains the training-score matrix, whereas the cross-product variant fits from the predictor cross-product and predictor-response cross-product without retaining training scores. fastPLS instead implements de Jong SIMPLS with deterministic IRLBA or approximate rSVD. The repeated CPU contract comprised float64 input, externally applied training centring, identical centred one-hot responses, identical splits and component counts, final held-out prediction, three fresh-process repetitions, and one effective thread. This repeated comparison used Breast, MetRef, and CIFAR-100. A separate single-run float32 feasibility extension evaluated the public IKPLS cross-product formulation on NMR and ImageNet, as detailed in Section S15.2. Resident-set size includes language-runtime allocation and is therefore interpreted as a workflow-feasibility measurement.

Table S10f. Archived-release single-thread CPU end-to-end comparison with IKPLS. The score-explicit variant corresponds to IKPLS Algorithm 1 and the cross-product variant to IKPLS Algorithm 2. Time and IQR are fitting-plus-prediction seconds; resident-set-size values are MiB. fastPLS rSVD rows use version 0.99.25, oversampling 20, two power iterations, seed 123, and case-specific diagnostics.

Dataset	Workflow	Accuracy (%)	Median time	Time IQR	Peak RSS	Incremental RSS
Breast	IKPLS score-explicit (Alg. 1)	94.29	0.000601	0.000519	130.5	0.8
Breast	IKPLS cross-product (Alg. 2)	94.29	0.000593	0.000084	131.2	0.8

Dataset	Workflow	Accuracy (%)	Median time	Time IQR	Peak RSS	Incremental RSS
Breast	fastPLS SIMPLS/IRLBA (v0.99.25; deterministic)	94.29	0.005000	0.002000	186.0	1.2
Breast	fastPLS SIMPLS/rSVD (v0.99.25; os20/p2/s123; qualified)	94.29	0.005000	0.000500	181.8	1.1
CIFAR-100	IKPLS score-explicit (Alg. 1)	70.95	0.885829	0.035523	595.2	74.8
CIFAR-100	IKPLS cross-product (Alg. 2)	70.95	0.217595	0.005360	583.8	64.5
CIFAR-100	fastPLS SIMPLS/IRLBA (v0.99.25; deterministic)	70.77	3.414000	0.079500	1252.3	624.3
CIFAR-100	fastPLS SIMPLS/rSVD (v0.99.25; os20/p2/s123; qualified)	72.13	3.585000	0.064000	1178.1	581.0
Metref	IKPLS score-explicit (Alg. 1)	75.00	0.003331	0.000063	134.7	2.1
Metref	IKPLS cross-product (Alg. 2)	75.00	0.002979	0.000165	135.2	3.6
Metref	fastPLS SIMPLS/IRLBA (v0.99.25; deterministic)	77.00	0.019000	0.001000	190.2	3.7
Metref	fastPLS SIMPLS/rSVD (v0.99.25; os20/p2/s123; qualified)	77.00	0.033000	0.000500	189.0	5.2

Reproducibility. Archived CPU results are in

benchmark_results/frozen_release_0.99.25/ikpls_cross_language_cpu/. The benchmark used fastPLS 0.99.25, IKPLS 6.1.2, float64, three fresh processes, and fixed component counts of 10 (Breast), 22 (MetRef), and 50 (CIFAR-100). The earlier JAX/CUDA development comparison was not reproducible from the archived environment and is not retained as central evidence.

S15.2 Large-case float32 feasibility extension

The extension used IKPLS 6.1.2, its public NumPy cross-product formulation, float32 input and arithmetic, one effective CPU thread, and no conversion time inside model timing. NMR had 1,200 training and 321 held-out rows, 13,000 predictors, and 28,355 responses. ImageNet had 1,000,000 training and 281,167 held-out rows, 1,024 predictors, and 1,000 classes. ImageNet predictors and centred one-hot responses were prepared as float32 arrays before fitting; held-out prediction was blocked. The float32 centring and one-hot preparation required 26.36 s and is reported separately. NMR runs used a 10-GiB virtual-memory guard so that structurally infeasible coefficient paths failed safely rather than exhausting the workstation. Each configuration was measured once and is therefore exploratory.

Table S10g. IKPLS 6.1.2 float32 feasibility on the NMR and ImageNet case studies. Time excludes external conversion and centring; ImageNet preprocessing required an additional 26.36 s. Peak RSS is absolute complete-process resident-set size. NMR at 50 components was not run because the retained coefficient tensor alone would require 68.66 GiB.

Dataset	Components	Status	Fit (s)	Predict (s)	Metric	Peak RSS (MiB)
NMR	1	Degenerate	53.13	3.45	RMSD 0.002113	4,281
NMR	5	Memory guard	-	-	Not estimated	-
NMR	50	Not run	-	-	B tensor 68.66 GiB	-
ImageNet	100	Success	43.98	8.69	Top-1 62.91%; top-5 86.40%	8,239
ImageNet	200	Success	62.21	6.19	Top-1 70.22%; top-5 91.36%	8,634
ImageNet	500	Success	109.93	7.16	Top-1 76.61%; top-5 94.64%	9,783
ImageNet	1,000	Success	190.84	6.22	Top-1 79.99%; top-5 95.61%	11,792

The NMR one-component run emitted the IKPLS near-zero-weight warning and produced predictions identical to the training-response mean; its RMSD is therefore a baseline value rather than evidence of a predictive latent component. The five-component attempt failed safely when a 13,000 x 13,000 float32 allocation could not be made under the guard. Public IKPLS retains a coefficient tensor with dimensions components x predictors x responses, corresponding to 1.373 GiB at one component, 6.866 GiB at five components, and 68.66 GiB at 50 components. On ImageNet, all four runs succeeded and closely tracked the archived fastPLS argmax accuracy trajectory. Runtime remains a software-workflow comparison rather than an estimator-matched comparison because IKPLS and SIMPLS differ mathematically and the archived fastPLS timing represents one maximal 1,000-component CUDA fit shared across prefixes.

Reproducibility. Scripts are in benchmark/ikpls_cross_language/. Raw logs, per-configuration CSV files, preprocessing time, and the joined comparison are in benchmark_results/ikpls_large_float32_20260826/. The

experiment used IKPLS 6.1.2, NumPy float32 arrays, one CPU thread, blocked ImageNet prediction, and conversion outside timing.

S16. Selected-point CPU and CUDA benchmark

This focused archived-release analysis uses identical stored splits and component counts for paired CPU/CUDA SIMPLS-rSVD routes: MetRef 22, Retina 20, and CIFAR-100 100. Every row uses fastPLS 0.99.25, oversampling 20, two power iterations, seed 123, case-specific diagnostics, and three repetitions. Host memory is the post-prediction minus pre-fit RSS snapshot; device memory includes runtime/context allocation.

Table S11. Archived-release paired CPU/CUDA SIMPLS-rSVD results. Every row uses fastPLS 0.99.25, oversampling 20, two power iterations, seed 123, and qualified-approximate status; time is median (IQR) over three runs.

Dataset	Family	A	Setting	Metric	Time median (IQR), s	Host RSS delta MiB	GPU delta MiB	Status
CIFAR-100	SIMPLS	100	fixed paired point	accuracy 0.8742	8.069 (0.127)	1.8	NA	CPU; v0.99.25; os20/p2/s123; qualified
CIFAR-100	SIMPLS	100	fixed paired point	accuracy 0.8739	0.724 (0.011)	99.3	537.0	CUDA; v0.99.25; os20/p2/s123; qualified
MetRef	SIMPLS	22	fixed paired point	accuracy 0.7700	0.034 (0.005)	6.0	NA	CPU; v0.99.25; os20/p2/s123; qualified
MetRef	SIMPLS	22	fixed paired point	accuracy 0.7700	0.267 (0.006)	345.2	204.0	CUDA; v0.99.25; os20/p2/s123; qualified
Retina	SIMPLS	20	fixed paired point	accuracy 0.9676	0.080 (0.003)	21.0	NA	CPU; v0.99.25; os20/p2/s123; qualified
Retina	SIMPLS	20	fixed paired point	accuracy 0.9676	0.264 (0.004)	348.1	213.0	CUDA; v0.99.25; os20/p2/s123; qualified

Selected CPU and CUDA SIMPLS-rSVD workflows
fastPLS 0.99.25; oversampling 20, two power iterations, seed 123

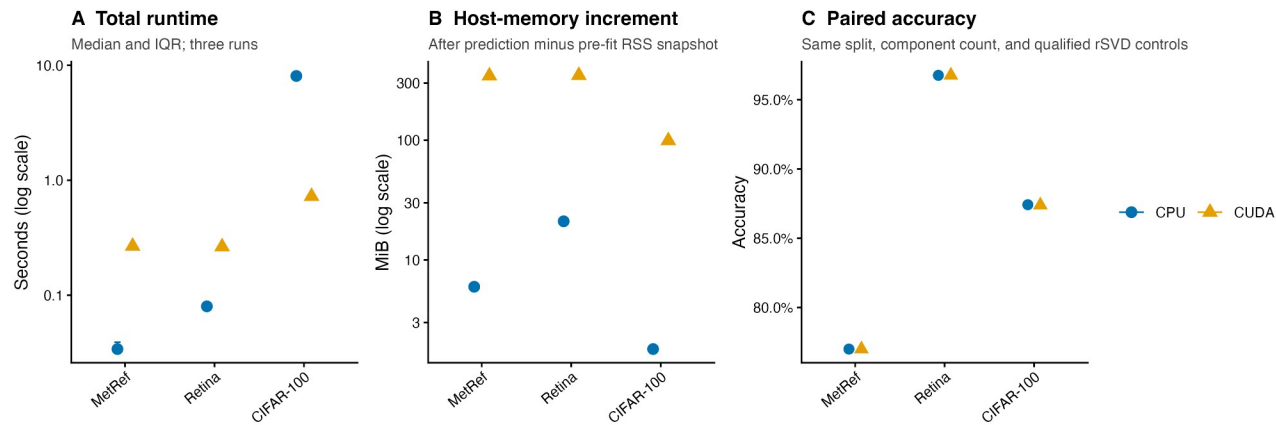


Figure S4. Frozen-release paired CPU/CUDA SIMPLS-rSVD workflows for MetRef, Retina, and CIFAR-100. Panels show total runtime, the post-prediction minus pre-fit host-RSS snapshot, and held-out accuracy. Points and IQRs summarize three runs. Every route used fastPLS 0.99.25, oversampling 20, two power iterations, seed

123, and identical splits and component counts. Device/context allocation is included; host-memory differences are not isolated solver workspace.

S17. NMR case study

The family-selected predictive analysis and paired backend analysis answer different questions. Component selection used five paired training-only splits. The PLS-SVD candidate grid was 1, 2, 3, 5, 7, 10, 25, 50, 75, 100, 125, 150, 165, 175, 200, 250, and 300 components. The SIMPLS candidate grid was 10, 25, 50, 75, 100, 125, 150, 165, 175, 200, 250, and 300 components. Both selected values were interior. This extended grid is the principal biomedical sensitivity analysis; the cross-dataset boundary rows remain fixed computational workloads.

For family f , component count a , and split $s = 1, \dots, 5$, let $e_{f,s}(a)$ denote validation RMSD. The implementation computes $m_f(a) = (1/5) \sum_s e_{f,s}(a)$, chooses the smallest $a_{\min}$ in $\operatorname{argmin}_a m_f(a)$, and defines the one-standard-error threshold $h_f = m_f(a_{\min}) + \operatorname{sd}_s[e_{f,s}(a_{\min})]/\sqrt{5}$, where sd is the sample standard deviation across the five splits. The eligible set is $E_f = \{a: m_f(a) \leq h_f\}$, and the returned count is $a_{\text{selected}} = \min(E_f)$.

For PLS-SVD, $a_{\min} = 5$, $h_f = 0.001528979123$, and $E_f = \{5\}$; therefore $a_{\text{selected}} = 5$. For SIMPLS, $a_{\min} = 50$, $h_f = 0.0009565399754$, and $E_f = \{50, 75, 100\}$; therefore $a_{\text{selected}} = 50$. Each value is selected within the evaluated grid, not asserted to be a global optimum.

The family-selected paired analysis compares CPU IRLBA, CPU rSVD, and CUDA rSVD at five PLS-SVD or 50 SIMPLS components. The 4.6-4.8 ppm interval was defined a priori from predictor chemical-shift labels and set to zero in both training and test predictors before any training-only split to suppress residual water resonance; no response coordinate was masked, and no held-out response information informed preprocessing. The deposited 165-component workflow remains historical context. Figure 4 displays AMI-0030-9 (index 38), selected as the held-out sample closest to the median 50-component SIMPLS CUDA/rSVD per-spectrum RMSD.

Table S12. Archived-release NMR evidence. Approximate rows use fastPLS 0.99.25, oversampling 20, two power iterations, seed 123, and case-specific diagnostics; deterministic rows use CPU IRLBA. Family-selected

prediction, the fixed-100-component implementation comparison, and the deposited historical workflow answer different questions.

Analysis	Family	Implementation	A	Predictive metric	Time s	Delta host MiB	Delta GPU MiB	Error detail / scope
family selected	PLS-SVD	CPU / RSVD	5	RMSD 0.0010426; Q ² 0.989159	4.574	post-fit 5.4	not captured	v0.99.25; rSVD os20/p2/s123; qualified
family selected	PLS-SVD	CUDA / RSVD	5	RMSD 0.0010426; Q ² 0.989159	0.422	post-fit 0.0	not captured	v0.99.25; rSVD os20/p2/s123; qualified
family selected	SIMPLS	CPU / RSVD	50	RMSD 0.0007560; Q ² 0.994301	152.092	post-fit 0.0	not captured	v0.99.25; rSVD os20/p2/s123; qualified
family selected	SIMPLS	CUDA / RSVD	50	RMSD 0.0007561; Q ² 0.994299	3.909	post-fit 0.0	not captured	v0.99.25; rSVD os20/p2/s123; qualified
fixed 100 components	PLS-SVD	CPU / RSVD	100	RMSD 0.0007208; Q ² 0.994819	23.703	post-fit 77.8	not captured	v0.99.25; rSVD os20/p2/s123; qualified
fixed 100 components	SIMPLS	CPU / RSVD	100	RMSD 0.0008074; Q ² 0.993498	307.484	post-fit 0.0	not captured	v0.99.25; rSVD os20/p2/s123; qualified
fixed 100 components	PLS-SVD	CUDA / RSVD	100	RMSD 0.0007317; Q ² 0.994660	1.207	post-fit 0.0	not captured	v0.99.25; rSVD os20/p2/s123; qualified
fixed 100 components	SIMPLS	CUDA / RSVD	100	RMSD 0.0008250; Q ² 0.993212	7.570	post-fit 0.0	not captured	v0.99.25; rSVD os20/p2/s123; qualified
family selected	PLS-SVD	CPU / IRLBA	5	RMSD 0.0010426; Q ² 0.989159	262.860	post-fit 0.0	not captured	v0.99.25; CPU IRLBA; deterministic
family selected	SIMPLS	CPU / IRLBA	50	RMSD 0.0007561; Q ² 0.994299	692.209	post-fit 0.0	not captured	v0.99.25; CPU IRLBA; deterministic
historical deposited workflow	PLS-SVD	deposited IRLBA	165	RMSD 0.0007099	447.601	peak 3605.5	NA	historical context; different workflow/hardware

S18. Historical ImageNet foundation-model embedding stress test

The historical archive contained 1,281,167 precomputed DINOv2 embeddings with 1,024 features and 1,000 class labels. Seed 123 assigned 1,000,000 rows to training and 281,167 to a noncanonical holdout that had informed earlier component choices. The exact checkpoint, pooling rule, extraction script, and auditable image-to-row mapping were unavailable. Archived fastPLS 0.99.25 reran only downstream label-aware float32 SIMPLS fitting and blocked prediction using CUDA rSVD oversampling 20, two power iterations, and seed 123. All values are single-run historical feasibility estimates; they are not used for biomedical, representation-quality, or comparative predictive claims. The accuracy, runtime, and memory visualization is presented as main-text Figure 5.

A separate repository benchmark uses FAISS [36] to compare nearest-neighbour retrieval on raw embeddings, PCA scores, and PLS scores, including transformation, index construction, and query time. It is an external retrieval analysis rather than a fastPLS classifier and is not pooled with the SIMPLS-LDA classification experiment below.

Table S13. Historical archived-release ImageNet downstream classification. Every row uses fastPLS 0.99.25, CUDA rSVD oversampling 20, two power iterations, seed 123, and qualified-approximate controls. One shared

fit supplies all component prefixes; the 1,000-component row is a boundary stress point, not a selected optimum.

All rows are single-run, partially reproducible feasibility estimates.

Experiment	Head / representation	A/dim.	Top-1	Top-5	End-to-end s	Host RSS MiB	GPU MiB	Qualification
classification	ARGMAX	100	0.64047	0.86775	2150.06	9884	782	v0.99.25; os20/p2/s123; qualified
classification	ARGMAX	200	0.70520	0.91380	2150.06	9884	782	v0.99.25; os20/p2/s123; qualified
classification	ARGMAX	300	0.73284	0.93098	2150.06	9884	782	v0.99.25; os20/p2/s123; qualified
classification	ARGMAX	400	0.75233	0.94048	2150.06	9884	782	v0.99.25; os20/p2/s123; qualified
classification	ARGMAX	500	0.76643	0.94621	2150.06	9884	782	v0.99.25; os20/p2/s123; qualified
classification	ARGMAX	600	0.77801	0.95041	2150.06	9884	782	v0.99.25; os20/p2/s123; qualified
classification	ARGMAX	700	0.78707	0.95276	2150.06	9884	782	v0.99.25; os20/p2/s123; qualified
classification	ARGMAX	800	0.79332	0.95438	2150.06	9884	782	v0.99.25; os20/p2/s123; qualified
classification	ARGMAX	900	0.79736	0.95538	2150.06	9884	782	v0.99.25; os20/p2/s123; qualified
classification	ARGMAX	1000	0.79980	0.95613	2150.06	9884	782	v0.99.25; os20/p2/s123; qualified
classification	LDA	100	0.78061	0.92779	1225.33	9718	4792	v0.99.25; os20/p2/s123; qualified
classification	LDA	200	0.79073	0.93201	1225.33	9718	4792	v0.99.25; os20/p2/s123; qualified
classification	LDA	300	0.79548	0.93344	1225.33	9718	4792	v0.99.25; os20/p2/s123; qualified
classification	LDA	400	0.79844	0.93394	1225.33	9718	4792	v0.99.25; os20/p2/s123; qualified
classification	LDA	500	0.80067	0.93468	1225.33	9718	4792	v0.99.25; os20/p2/s123; qualified
classification	LDA	600	0.80297	0.93550	1225.33	9718	4792	v0.99.25; os20/p2/s123; qualified
classification	LDA	700	0.80508	0.93675	1225.33	9718	4792	v0.99.25; os20/p2/s123; qualified
classification	LDA	800	0.80689	0.93787	1225.33	9718	4792	v0.99.25; os20/p2/s123; qualified
classification	LDA	900	0.80847	0.93869	1225.33	9718	4792	v0.99.25; os20/p2/s123; qualified
classification	LDA	1000	0.80938	0.93929	1225.33	9718	4792	v0.99.25; os20/p2/s123; qualified

S19. Archival repeated outer-partition sensitivity analysis

This development-stage sensitivity analysis was not regenerated from the archived 0.99.25 release and is excluded from central release claims. It is retained to document variability observed during method development. MetRef, GTEx v8, and Retina used ten stratified 80/20 outer partitions with five-fold training-only component selection; NMR used five random 80/20 partitions and three-fold selection. The empirical ranges are descriptive, not confidence intervals for the archived-release analyses.

Table S14. Supporting repeated-partition predictive dispersion and component-selection sensitivity. These development results show training-partition variability but were not regenerated from the archived 0.99.25 release and are not used as central release claims. Selection frequencies at a boundary are reported explicitly.

Dataset	Family	Head	Outer n	Metric	Mean (SD)	Empirical range	Selected A range	Upper-bound freq.	Constraint
gtex v8	kernel PLS	argmax	10	accuracy	0.963437 (0.0048)	0.956072-0.968702	100-100	1.00	

Dataset	Family	Head	Outer n	Metric	Mean (SD)	Empirical range	Selected A range	Upper-bound freq.	Constraint
gtex v8	OPLS	argmax	10	accuracy	0.961628 (0.00435)	0.955362-0.96741	100-100	1.00	
gtex v8	PLS-SVD	argmax	10	accuracy	0.942765 (0.00642)	0.929554-0.949031	31-31	1.00	rank constrained
gtex v8	SIMPLS	argmax	10	accuracy	0.963437 (0.0048)	0.956072-0.968702	100-100	1.00	
metref	kernel PLS	argmax	10	accuracy	0.748023 (0.0269)	0.706921-0.789689	22-22	1.00	
metref	kernel PLS	lda	10	accuracy	0.858757 (0.0341)	0.792938-0.900141	22-22	1.00	
metref	OPLS	argmax	10	accuracy	0.756497 (0.0245)	0.71822-0.792797	22-22	1.00	
metref	OPLS	lda	10	accuracy	0.863842 (0.033)	0.813559-0.901412	22-22	1.00	
metref	PLS-SVD	argmax	10	accuracy	0.742938 (0.0222)	0.713136-0.77274	20-21	0.80	rank constrained
metref	PLS-SVD	lda	10	accuracy	0.821469 (0.0269)	0.76596-0.846186	21-21	1.00	rank constrained
metref	SIMPLS	argmax	10	accuracy	0.748023 (0.0269)	0.706921-0.789689	22-22	1.00	
metref	SIMPLS	lda	10	accuracy	0.858757 (0.0341)	0.792938-0.900141	22-22	1.00	
nmr	PLS-SVD	regression	5	RMSD	0.000798261 (7.43e-05)	0.000711782- 0.0008999	50-100	0.20	
nmr	SIMPLS	regression	5	RMSD	0.000773708 (5.63e-05)	0.000701997- 0.000845392	50-50	0.00	
retina	kernel PLS	argmax	10	accuracy	0.96799 (0.00151)	0.965369-0.970341	20-50	0.10	
retina	kernel PLS	lda	10	accuracy	0.968737 (0.00152)	0.965966-0.970937	20-50	0.10	
retina	OPLS	argmax	10	accuracy	0.951283 (0.00211)	0.947217-0.953131	30-40	0.00	
retina	OPLS	lda	10	accuracy	0.967143 (0.00152)	0.964131-0.96898	30-50	0.10	
retina	PLS-SVD	argmax	10	accuracy	0.926266 (0.0022)	0.923146-0.930036	11-11	1.00	rank constrained
retina	PLS-SVD	lda	10	accuracy	0.947457 (0.00204)	0.944398-0.950206	11-11	1.00	rank constrained
retina	SIMPLS	argmax	10	accuracy	0.96799 (0.00151)	0.965369-0.970341	20-50	0.10	
retina	SIMPLS	lda	10	accuracy	0.968737 (0.00152)	0.965966-0.970937	20-50	0.10	

S20. Analysis provenance

All central rows resolve to archived fastPLS 0.99.25, Git commit

7887401b09e25f54a546a253c255741cb1ab48e5, and source archive SHA-256

604f74d72f5e9540f7378efd37f2ca6ed87ccb9e73b912211331a8cd97233481. The machine-readable ledger contains SHA-256 values for 185 evidence files. The deposited 165-component NMR workflow is the sole historical context row in the central ledger.

Table S15. Archived-release central analysis provenance. Archive paths are relative to the repository root; full per-file checksums are in provenance/frozen_evidence_file_ledger.csv.

ID	Authoritative output	Result archive	Version	Source status	Generating script	SHA-256 prefix
F01	Execution archive	publication_release_0.99.25/	0.99.25	7887401b09e2	frozen_release_manifest.tsv	604f74d72f5e
F02	Regression contracts	benchmark_results/ frozen_release_0.99.25/ regression_contract_*	0.99.25	7887401b09e2	run_release_regression_contract. R	604f74d72f5e
F03	SIMPLS estimator validation	benchmark_results/ frozen_release_0.99.25/ simpls_estimator_preservation	0.99.25	7887401b09e2	run_simpls_estimator_preservati on.R	604f74d72f5e
F04	Controlled scaling	benchmark_results/ frozen_release_0.99.25/ controlled_scaling	0.99.25	7887401b09e2	controlled_scaling/run_grid.R	604f74d72f5e
F05	External SIMPLS comparison	benchmark_results/ frozen_release_0.99.25/ external_simpls	0.99.25	7887401b09e2	external_simpls_timing/ run_benchmark.R	604f74d72f5e

ID	Authoritative output	Result archive	Version	Source status	Generating script	SHA-256 prefix
F06	IKPLS CPU comparison	benchmark_results/ frozen_release_0.99.25/ ikpls_cross_language_cpu	0.99.25	7887401b09e2	ikpls_cross_language/ run_benchmark.py	604f74d72f5e
F07	Selected CPU/CUDA	benchmark_results/ frozen_release_0.99.25/ selected_backend	0.99.25	7887401b09e2	run_selected_backend.R	604f74d72f5e
F08	NMR	benchmark_results/ frozen_release_0.99.25/nmr	0.99.25	7887401b09e2	run_nmr_frozen.R	604f74d72f5e
F09	ImageNet downstream PLS fitting and prediction	benchmark_results/ frozen_release_0.99.25/ imagenet	0.99.25	7887401b09e2 downstream rerun: historical embedding extraction incomplete	benchmark_imagenet_qualified_ top5_path.R	604f74d72f5e
H01	Deposited NMR workflow	benchmark_results/ manuscript_revision_cycle64_2 0260726	historical	deposited source	FastPLS (1).R	historical
F10	Exact dense SIMPLS reference	benchmark_results/ frozen_release_0.99.25/ simpls_exact_reference	0.99.25	Frozen package; post-freeze audit script 0ef55efd44d	benchmark_simpls_exact_refere nce.R	0b38a7690db6

S21. Unrestricted synthetic end-to-end reproduction

benchmark/synthetic_end_to_end_example.R reproduces the complete unrestricted workflow. The minimal user example below shows deterministic fitting, the primary rSVD route, prediction, diagnostic inspection, CUDA dispatch, and informative handling of an unavailable backend.

```
remotes::install_github("tkcaccia/fastPLS", ref = "7887401b09e25f54a546a253c255741cb1ab48e5")

library(fastPLS)

data(breast)

fit_det <- pls(breast$X_train, breast$y_train, ncomp = 1:5,
              method = "simpls", svd.method = "irlba", backend = "cpu")

fit_rsvd <- pls(breast$X_train, breast$y_train, ncomp = 1:5,
              method = "simpls", svd.method = "rsvd", backend = "cpu",
              oversample = 20, power = 2, seed = 123)

fit_rsvd$diagnostics[c("solver", "status", "approximation_audited")]
fit_rsvd$diagnostics$rsvd[c("oversample", "power", "case_audit")]

pred <- predict(fit_rsvd, breast$X_test)
pred_5 <- pred$Ypred[["ncomp=5"]]

evaluate(predicted = pred_5, observed = breast$y_test)

if (has_cuda()) {
  fit_cuda <- pls(breast$X_train, breast$y_train, ncomp = 1:5,
                method = "simpls", svd.method = "rsvd", backend = "cuda")

  list(residency = fit_cuda$gpu_resident, diagnostics = fit_cuda$diagnostics)
} else {
  try(pls(breast$X_train, breast$y_train, backend = "cuda"))
}
```

S22. Repository-only detailed material

Full component paths, raw run rows, sensitivity analyses, and machine-level diagnostics are indexed in `benchmark/MANUSCRIPT_EVIDENCE_ARCHIVE.md`. Their CSV, RDS, PDF, PNG, log, and session-information files remain available for reproducibility without duplicating the primary tables in this supplement.

S23. Continuous-integration and platform-test status

The public BiocStaging build for fastPLS 0.99.25 used commit `7887401b09e25f54a546a253c255741cb1ab48e5` and completed successfully (<https://github.com/r-universe/biocstaging/actions/runs/32822192644>). Hosted macOS jobs establish compilation and CPU fallback, not Metal runtime execution. Linux CUDA and macOS Metal runtime validation therefore remains tied to dedicated hardware reports. Unit tests verify informative failure before model allocation for unavailable backend/solver combinations. Public package metadata: <https://biocstaging.r-universe.dev/fastPLS>.

Supplementary references

References are numbered as in the main manuscript. The final mapping is Retina [25], Tabula Muris [26], PRISM [27], ImageNet [28], DINOv2 [29], CIFAR-100 [30], UNI [31], Prov-GigaPath [32], Improved Kernel PLS [33], IKPLS [34], fast cross-product validation [35], and FAISS [36].